Article

The International Journal of
Robotics Research
2014, Vol. 33(13) 1683–1708
© The Author(s) 2014
Reprints and permissions:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/0278364914547786
ijr.sagepub.com

Sampling-based A* algorithm for robot path-planning

Sven Mikael Persson and Inna Sharf

Abstract

This paper presents a generalization of the classic A algorithm to the domain of sampling-based motion planning. The root assumptions of the A* algorithm are examined and reformulated in a manner that enables a direct use of the search strategy as the driving force behind the generation of new samples in a motion graph. Formal analysis is presented to show probabilistic completeness and convergence of the method. This leads to a highly exploitative method which does not sacrifice entropy. Many improvements are presented to this versatile method, most notably, an optimal connection strategy, a bias towards the goal region via an Anytime A* heuristic, and balancing of exploration and exploitation on a simulated annealing schedule. Empirical results are presented to assess the proposed method both qualitatively and quantitatively in the context of high-dimensional planning problems. The potential of the proposed methods is apparent, both in terms of reliability and quality of solutions found.*

Keyword

Sampling-based algorithm, path-planning, motion-planning, A*, RRT*, probabilistic completeness, simulated annealing, robot manipulator, high-dimensional planning, optimization

1. Introduction

Complex path-planning problems typically involve high-dimensional configuration spaces, complex collision-detection geometries, kinematic or dynamic constraints, uncertain or dynamic environments, unconventional distance metrics or cost functions, and other domain-specific considerations. These types of problems are generally very difficult to solve by a direct trajectory optimization, and are often inadequately served by simple navigation heuristics (e.g. potential field approaches). Deterministic algorithms that rely on a discretization of the configuration space to represent all possible moves the robot can make are usually computationally too expensive and dedicate far too much effort, in time and memory, where none is needed to meet the goal of finding a feasible and optimal path. For these reasons, sampling-based motion-planning methods have become a very popular alternative as they probabilistically construct a motion graph to serve the purposes of the planning problem and can thus be controlled to direct resources more effectively.

Sampling-based motion planning started to gather serious traction with the introduction of two families of methods: the *rapidly-exploring random tree* (RRT) (LaValle, 1998; LaValle and Kuffner, 2001); and the *probabilistic roadmap* (PRM) (Kavraki et al., 1996; Kavraki and Latombe, 1998). The key development with the introduction of the RRT algorithm is the concept of the *Voronoi bias* (sometimes called *Voronoi pull*) which drives the creation of new samples through an expansion of existing samples towards unexplored regions represented by random samples drawn from the overall configuration space (most likely, unexplored regions). The first important modification to the RRT algorithm was the introduction of a bidirectional strategy, i.e. growing two trees, one from the goal and one from the start, and greedily attempting to connect them (Kuffner and LaValle, 2000). Since then, countless other modifications have been introduced, ranging from general to domain-specific.

The PRM algorithm is quite different in nature as it attempts to create a complete representation of the collision-free configuration space through a roadmap of samples that are all connected as one graph (Kavraki et al., 1996). For early overviews and summaries of PRM algorithms, variations, and comparative studies, the reader is directed towards (Geraerts and Overmars, 2002; Sucan and

McGill University, McConnell Engineering Building, Montreal, Quebec, Canada

Corresponding author:
Sven Mikael Persson, McGill University, McConnell Engineering Building, 3480 University Street, Montreal, Quebec, H3A 2A7, Canada.
Email: mikael.persson@mail.mcgill.ca

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Kavraki, 2010). The important concepts introduced include mainly the formal analysis of probabilistic completeness and of space connectivity, as well as the more practical concept of monitoring the density of nodes in the motion-graph and leveraging that information to "push" the expansion in useful regions.

This last concept will be critical in the present paper. Another early algorithm that builds on this idea is the *expansive spaces tree* (EST) (Hsu et al., 1999) which generates a tree via random walks (or expansions) of existing nodes in the tree, which are picked with a probability inversely proportional to the number of nodes in their direct neighborhood. The formal analysis in the present paper will rely heavily on the analysis of the EST algorithm, as presented in the thesis (Hsu, 2000). Few later algorithms directly fall under this category of algorithms, but two notable later developments are: the *single-query bi-directional lazy* planner (SBL) (Sanchez and Latombe, 2001) which introduces many practical improvements to the EST algorithm; and the Guided-EST algorithm (Phillips et al., 2004) which introduces a more sophisticated density heuristic which incorporates, among other things, the A* cost of the nodes. In that sense, the algorithm presented in this paper arrives at a similar result, but with the critical difference that the density and A* cost combination are derived from a generalization of the A* algorithm itself and are justified by formal analysis.

Another important early development in sampling-based motion planning was the realization that quasi-random sampling could, in general, be sufficient and beneficial to such algorithms (Branicky et al., 2001). By their nature, quasi-random samplers rely on a finite discretization of space, with controlled interval sizes, and generate samples from the finite set which tends to produce more uniform distributions. In other words, this can be seen as a probabilistic discovery of a finite motion graph, avoiding the problem of representing the complete motion graph in memory or having to traverse it entirely, while benefiting from its limited and uniform density, another important concept in the present paper.

The present paper inscribes itself into the trend of bringing useful concepts from deterministic path planning into sampling-based methods. The classic deterministic path-planning method is the *A* algorithm* (Hart et al., 1968) which relies on a best-first exploration of the motion-graph to find an optimal path from a starting node to a goal node. Then, two relatively recent modifications to this classic algorithm have been made. First, the *Anytime A** algorithm was developed to speed up the initial search for a feasible path and then progressively improve it, and thus, achieving an *anytime* behavior (Likhachev et al., 2003). Then, dynamic reconfiguration of the motion-graph was added to create the *Anytime Dynamic A** (AD*) algorithm (Likhachev et al., 2005).

Some attempts have been made to incorporate these aforementioned deterministic algorithms more intimately with different kinds of sampling-based algorithms. For example, the strategies for the dynamic reconfiguration of the motion-graph were applied in a straight-forward manner to an RRT planner (Ferguson et al., 2006). Similarly, since the PRM algorithm requires a shortest-path algorithm to resolve a query once a sufficiently connected roadmap is available, one can alternate the generation of the motion graph and an anytime dynamic search through it, as in (Berg et al., 2006). Yet another possibility is to use a layered approach such as super-imposing a coarse discrete search and a sampling-based planner (Plaku, 2012).

More interestingly and more relevant to the present paper is the concept of expanding the motion graph through the shortest-path algorithm itself. One interesting example is the *Flexible Anytime Dynamic PRM* (FADPRM) (Belghith et al., 2006) which uses a mixture of the density metric (from PRM) and the AD* heuristic to select nodes to be expanded by random walks. The FADPRM relies on many heuristics and fine-tuned parameters that make its application rather difficult in practice. Another notable example of driving the sampling with a shortest-path heuristic is the *Utility-guided RRT* (Burns and Brock, 2007). This method is especially relevant to the present paper because it attempts to model the utility of the exploration in a probability theoretic fashion. Building off a prior method by the same authors (Burns and Brock, 2005) which explored the idea of predictive statistical models to learn the topology of the free configuration-space, the Utility-guided RRT uses local predictions of the utility of exploring around a given node and try to sample along the least-explored directions therefrom. Similarly, the application of equivalence classes as a means to exhaust neighborhoods is another predictive model that can be applied (Gonzalez and Likhachev, 2011). In the present paper, we rely on a similar idea by relying on the expected value of the total path cost (as in the A* algorithm) with the aim of maximizing exploitation by sampling around optimal regions.

Coming back to the classical sampling-based approaches, Karaman and Frazzoli (2011) have presented a very influential paper in which they describe three new algorithms, RRT*, *rapidly-exploring random graph* (RRG), and PRM*, which are proven to be asymptotically optimal. The RRT* is probably the most widely used sampling-based algorithm today and varies from the RRT mainly in the fact that it keeps track of the accumulated travel cost and performs optimal re-wirings to conserve a record of the optimal path from the root to any other node. A branch-and-bound strategy can also be added to the algorithm (Karaman et al., 2011). The argument behind the branch-and-bound strategy is that nodes that cannot contribute to optimal paths (from start to goal) should be pruned from the motion tree to reduce wasted efforts.

In machine learning and numerical optimization literature in particular, there is a recurring theme, that of *exploration* versus *exploitation*. This is the problem of choosing between broadening a search in order to discover all possible solutions versus refining the current best solution(s) (e.g. gradient descent). This balancing act has been less

Downloaded from ijr.sagepub.com by guest on January 20, 2015

present in sampling-based motion-planning literature, which has implicitly favored exploration since its inception (Kavraki et al., 1996; LaValle, 1998) and sometimes explicitly (Sucan and Kavraki, 2012), but there are notable exceptions. In many practical implementations of unidirectional RRT methods, the goal region is sampled on a regular basis in a greedy attempt to grow the tree more rapidly towards it. The *exploration-exploitation tree* (EET) (Rickert et al., 2008) is a method which draws samples from a region around the goal, making incremental expansions towards the obtained sample, and expanding or shrinking the sampling region according to a heuristic based on the success rate of the expansions, thus balancing exploration and exploitation via a larger or smaller sampling region around the goal.

Another interesting method, presented in Vazquez-Otero et al. (2012), uses a dynamic reaction–diffusion process to expand a search for the goal location and then contract while leaving a simulated tension between the start and goal, resulting in an optimal path. This strategy is strangely reminiscent of *simulated annealing* methods used in numerical optimization and certain machine learning algorithms. The authors did not find any reported use of simulated annealing in a sampling-based motion planner, but it has shown to be useful when solving a path-planning problem as a general non-linear trajectory-optimization problem. Most recently, the *covariant Hamiltonian optimization for motion planning* (CHOMP) method presents one state-of-the-art use of simulated annealing in motion planning (Zucker et al., 2013). We mention these methods mainly because one of the central novelty in the present paper is the application of a simulated annealing strategy to balance exploration and exploitation in the proposed sampling-based motion planner.

This paper is organized as follows. Section 2 presents a generalization of the A* algorithm by deconstructing its basal assumptions and casting them in a probability theoretic framework that can be used with local predictive models to drive the expansion of a motion graph. We name this method the *sampling-based A** algorithm (SBA*) and provide formal analysis to characterize its convergence. Then, in Section 3, the practical SBA* algorithm is presented with a number of refinements to it, notably, an optimal connection strategy, an anytime heuristic to provide a stronger goal-bias, and the use of simulated annealing to balance exploration and exploitation. Finally, in Section 4, results are presented to characterize the behavior of the proposed algorithm in a cluttered environment, in high-dimensional spaces, and for the practical application motivating this work: motion planning for a seven-degree-of-freedom (7-dof) manipulator to capture a free-floating target.

2. Generalizing the A* algorithm

This section outlines the process of generalization of the A* algorithm (Hart et al., 1968) to a sampling-based path-planning approach. The A* search strategy has a number of advantages, two of which are lacking in current sampling-based approaches, that is, the search is focused on the current most promising path and the first solution found is the optimal one (within the discretization). However, the A* search is an exhaustive search over a finite motion graph (e.g. a "grid"), which is problematic since, by definition, a sampling-based approach involves the construction of the motion-graph through sampling of the configuration space, and can thus be generated *ad infinitum*. We tackle this problem by examining the underlying rationale behind the A* algorithm, and we generalize that rationale, through local predictive statistical models, so that it is suitable for a sampling-based approach.

2.1 Overview of A*

At the core of the A* search algorithm is a priority queue which chooses the most promising node to visit. Visiting a node entails surveying its neighboring nodes, adding them to the priority queue as needed, and the process is repeated until the goal node is discovered. In other words, nodes of the motion graph start with the label "undiscovered", then become "discovered but not expanded" (or "open"), and finally, become "expanded" (or "closed"). Needless to say, the search strategy is very simple, yet very effective, in fact, optimal under most usual conditions.

Clearly, the algorithm is driven by a measure of how "promising" the visit of a node is to the search. This measure is obtained through an approximation that underestimates the total travel cost when taking a path through a given node. In concrete terms, given a node u, if we accumulate the cost to travel from the start to that node via the shortest path through the discovered portion of the motion-graph, denoting that cost as $g(u)$, and then compute a heuristic value for the remaining cost to the goal, denoting it as $h(u)$, we can obtain a lower-bound approximation of the total cost as

$$f(u) \equiv g(u) + h(u) \tag{1}$$

where the heuristic value $h(u)$ is required to be equal to or less than the actual travel cost along a non-colliding path from u to the goal node. Usually, $h(u)$ is simply the "bird-flight" distance, i.e. the distance between u and the goal node in the configuration space when obstacles are ignored. Once the total travel cost can be approximated, the node with the least total travel cost is considered as the most promising and, thus, the priority queue chooses the minimum element (i.e. a "min-heap"). The optimality of the A* search hinges upon requirements on the heuristic value, most notably, that it does not overestimate the remaining cost (called *admissibility*) and that it is monotonically decreasing as progress is made towards the goal (called *consistency*). A "bird-flight" distance in a configuration space with a proper metric automatically satisfies these conditions, and since most path-planning applications, even

Downloaded from ijr.sagepub.com by guest on January 20, 2015

in the most esoteric domains, have these properties already for other reasons (e.g. having consistent neighborhoods), the conditions for optimality of the A* search are easily met.

The A* algorithm is well-known to most researchers in the field, and we will not provide further detail on it in this section. The key elements to keep in mind are, first, the idea of choosing the most "promising" nodes and, second, approximating the total travel cost by a combination of the accumulated cost from the start node and the heuristic evaluation of the remaining cost to reach the goal. The latter is simple to do and is available to virtually all application domains. The former, however, will need further dissection to make the A* search applicable as the driver for a sampling-based path-planning approach.

2.2 Generalization of the node value function

The classic formulation of the A* search algorithm (Hart et al., 1968) makes a silent assumption, that is, expanding a node is only useful if there is information to be gained from expanding it. This can seem like a trivial point, but it becomes important in a sampling-based approach. Given that the A* search is normally conducted on a finite motion graph, upon the first examination of a node, one can exhaustively survey its neighborhood, after which there is obviously no more information to be gained from that neighborhood, and the node can be "closed" and never expanded again.

If one were to simply take an A* search algorithm and replace the exhaustive survey of the neighborhood by the generation of samples within the nearby configuration space, the algorithm would make no progress at all, since it would repeatedly choose nodes from the same area and generate more samples in that neighborhood, *ad infinitum*. We need a measure of the information gain to be expected from generating a sample in a given neighborhood, that is, a measure of how exhaustively searched the neighborhood of a node is. In other words, we must depart from the binary concept of "open" versus "closed" nodes from the classic A* algorithm, and generalize the concept to a more appropriate model of information gain.

In more concrete terms, the A* search algorithm chooses the node u_i to expand next based on the following criteria:

$$u_i = \arg\min_u f(u) \quad \text{with } u \in \{\text{OPEN}\} \tag{2}$$

which could be reformulated by defining a new node value function:

$$r(u) \equiv \begin{cases} f(u) & \text{if } u \in \{\text{OPEN}\} \\ \infty & \text{otherwise} \end{cases} \tag{3}$$

$$u_i = \arg\min_u r(u) \tag{4}$$

which clearly exposes the binary nature of the expected information gain from expanding a node. In other words, the value of $r(u)$ reflects the expected total travel cost of a *new path* discovered that *goes through* node u. The emphasis here is on the fact that the partial path is *new*, and that it is *feasible*, both encoded by whether the node u belongs to the OPEN set or not, since a node will not be in the OPEN set if it is unreachable by a collision-free path or has already been expanded.

We define two types of events that can occur during a visitation.

$$\text{NEW}: \quad \text{A new partial path was constructed} \tag{5}$$

$$\text{FREE}: \quad \text{A collision - free segment was discovered} \tag{6}$$

This leads to the following redefinition of $r(u)$:

$$r(u) = P(\text{NEW, FREE}|u)\, f(u) + (1 - P(\text{NEW, FREE}|u))\infty \tag{7}$$

$$P(\text{NEW, FREE}|u) = \begin{cases} 1 & \text{if } u \in \{\text{OPEN}\} \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

where $P(\text{NEW,FREE}\ |u)$ is the probability that, given a node u, a visitation will yield a new and collision-free path, which, in a classic A* algorithm, is encoded by the OPEN set.

Clearly, $r(u)$ is an expected value from a binomial distribution, which we can express as such:

$$r(u) = E_{\mathcal{N}(u)}[f(u)] \tag{9}$$

which is the expected value of $f(u)$ in the neighborhood of u, when rejecting nodes that are not reachable through a collision-free path or have already been explored. This formulation is clearly far more general, and now holds more hope in transposing the A* search method to sampling-based path planning.

For any continuous probability $P(\text{NEW,FREE}\ |u)$, we obtain the following expression for $r(u)$:

$$r(u) = \frac{f(u)}{P(\text{NEW, FREE}|u)} \tag{10}$$

which, as expected, goes to infinity as the probability of discovering a new, collision-free node in the neighborhood $\mathcal{N}(u)$ goes to zero. In addition, we note that, given the definition of the NEW and FREE events, they are independent distributions, meaning that we can do the following:

$$P(\text{NEW, FREE}|u) = P(\text{NEW}|u)\, P(\text{FREE}|u) \tag{11}$$

In a classic A* search algorithm, any node belonging to the OPEN set is given *a priori* values of 1 for the probabilities $P(\text{NEW}\ |u)$ and $P(\text{FREE}\ |u)$, and given that a visitation is an exhaustive search in the finite neighborhood $\mathcal{N}(u)$, when the node is closed, those probabilities drop to 0. In a sampling-based approach, this open–close transition is progressively achieved as the neighborhood $\mathcal{N}(u)$ gets filled with nodes sampled from the configuration space.

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Algorithm 1. Randomized expansion from best sample.

```
1: Initialize a graph G = (V, E) with V = {u_init}.
2: repeat
3:    u_exp ← ChooseBestSample(V)                    ▷ Pick sample from V
4:    {u_new, success} ← RandomWalk (u_exp)          ▷ Generate sample near u_exp
5:    if success then
6:       V ← V ∪ {u_new}
7:       Connect u_new to G by adding appropriate edges to E.
8:    end if
9: until u_new in goal region
```

2.3 Idealized sampling-based A* algorithm

For analytical purposes, it is sufficient to assume a simplified algorithm, and therefore, Algorithm 1 presents a general random-expansion algorithm which is simply a re-statement of Algorithm 4.1 of Hsu (2000) with the sampling strategy substituted for the ChooseBestSample function. The RandomWalk function is a simple random steering function that attempts to produce a new sample u_{new} in the neighborhood of u_{exp} with a feasible path to it. Then, the algorithm integrates that new sample to the motion graph. Finally, the iterations stop when a sample is within the goal region.

This idealized algorithm is presented mainly for exposition purposes and to be used in the subsequent formal analysis, while the practical algorithm is presented in Section 3. The key aspect of this idealized algorithm is choosing the "best" sample. From the previous section, it should be apparent how we would choose such a best sample for expansion. Faithfully conforming to the classic A* search algorithm, we would choose the best sample to expand based on the best expected value of $r(u)$, as so

$$u_{exp} = \arg\min_{u \in V} r(u) \tag{12}$$

Broadly speaking, this algorithm captures the essential elements of the SBA* algorithm. Given the similarities with the EST algorithm (Hsu et al., 1999), we can consider that the SBA* falls under that family of methods. The similarities between these algorithms run deep enough, in fact, that it is natural to express the formal analysis of the idealized SBA* algorithm in terms of the formal analysis that supports the EST family of algorithms.

2.4 Formal analysis

Given the similarities between EST and SBA*, we start the formal analysis by building upon the work of Hsu (2000) on an idealized EST algorithm, to which we add a few relevant definitions and lemmas to build a proof of probabilistic completeness with a convergence rate of the idealized SBA* algorithm. In this section, we refer to the configuration space as $\mathcal{X}$ and its collision-free subset as $\mathcal{F} \subset \mathcal{X}$. Moreover, we use $\mathcal{R}_\ell(u) \subset \mathcal{F}$ to denote the region of space locally reachable from a point u, e.g. a hyperball around u or a region constrained by controllability limitations.

The analysis relies on the concept of *expansive* spaces (Hsu et al., 1999), and we begin with a brief reminder of two fundamental definitions relevant to this concept, as follows.

Definition 1 (β-LOOKOUT). Given a constant $\beta \in (0,1]$, the β-LOOKOUT of a set $\mathcal{S} \subset \mathcal{F}$ is

$$\beta(\mathcal{S}) \equiv \{u \in \mathcal{S} \mid \mu(\mathcal{R}_\ell(u) \setminus \mathcal{S}) \geq \beta\mu(\mathcal{R}_\ell(\mathcal{S}) \setminus \mathcal{S})\} \tag{13}$$

where $\mu(\mathcal{S})$ denotes the volume of a set $\mathcal{S}$ and $\beta(\mathcal{S})$ denotes its β-LOOKOUT.

Definition 2 ((α, β)-expansive). Given constants $\alpha, \beta \in (0,1]$, the set $\mathcal{F}$ is (α, β)-expansive if for every point $u \in \mathcal{F}$ and subset $\mathcal{S} \subset \mathcal{R}_\ell(u)$, we have that $\mu(\beta(\mathcal{S})) \geq \alpha\mu(\mathcal{S})$.

In less formal terms, the β-LOOKOUT denotes the region from which a significant portion of new space could be discovered by a random walk or local sampling, while an (α, β)-expansive space has the property of a lower bound on the volume of the lookout region around any arbitrary point. Together, these definitions form the basis for constructing sequentially reachable chains of milestones that explore the space (Hsu, 2000), making them an ideal basis for the formal analysis to follow.

2.4.1 Unbiased sampling. We begin the analysis by considering the case of unbiased sampling, that is, considering uniform exploration of the space. To start the construction of this formal analysis, we present a definition of the local Voronoi regions.

Definition 3 (Local Voronoi region). Given a point $u \in \mathcal{F}$ being the seed of a Voronoi region $\mathcal{V}(u)$, the *local Voronoi region* $\mathcal{V}_\ell(u)$ is defined as

$$\mathcal{V}_\ell(u) \equiv \mathcal{R}_\ell(u) \cap \mathcal{V}(u) \tag{14}$$

where $\mathcal{R}_\ell(u) \subset \mathcal{F}$ is the locally reachable region around u.

In other words, the local Voronoi region is the set of all points that are closer to u than to any other point and that are locally reachable from u. The local Voronoi region of a point set V is then $\mathcal{V}_\ell(V) \equiv \bigcup_{u \in V} \mathcal{V}_\ell(u)$.

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Lemma 1. *Given a metric space* $(\mathcal{X}, d)$ *and a set* V *of points* $u \in \mathcal{X}$ *for which there is local reachable region* $\mathcal{R}_\ell(u) = \{v \in \mathcal{X} \mid d(u,v) \leq R\}$, *the local Voronoi space of* V (*as per Definition 3*) *is coincident with the local reachable s*pace of V, i.e. $\mathcal{V}_\ell(V) = \mathcal{R}_\ell(V)$.

Proof. Since the Voronoi regions span the entire space $\mathcal{X}$, a point can only be excluded from $\mathcal{V}_\ell(V)$ if it is not reachable from the point that is closest to it (the seed of the Voronoi region), and because the local reachable regions $\mathcal{R}_\ell(u)$ are hyperballs of equal radius, such a point cannot be reachable from any other point in V, and thus, must necessarily be excluded from $\mathcal{R}_\ell(V)$ as well. That is, a point belongs to $\mathcal{V}_\ell(V)$ if and only if it belongs to $\mathcal{R}_\ell(V)$.

It is important to note that Lemma 1 extends to a free configuration space $\mathcal{F}$ over a metric space, as long as $\mathcal{F}$ is compact (cf. fully controllable state space). The proof is only more involved if one has to consider reachable regions $\mathcal{R}_\ell(u)$ that intersect the non-convex boundaries of $\mathcal{F}$. For that reason, we omit this generalization here.

Also, one should note that a consequence of Lemma 1 is that it presents an alternative to computing the volume of $\mathcal{R}_\ell(V)$ by taking the sum of the volumes of the local Voronoi regions:

$$\mu(\mathcal{R}_\ell(V)) = \sum_{v \in V} \mu(\mathcal{V}_\ell(v)) \tag{15}$$

which suggests a sampling strategy based on the fraction $\mu(\mathcal{V}_\ell(u))$ of the total volume $\mu(\mathcal{R}_\ell(V))$ as a basis for generating nearly uniform samples in that region.

Lemma 2. *Given a point set* V *where each point* $u \in \mathcal{F}$ *has a locally reachable region* $\mathcal{R}_\ell(u) \subset \mathcal{F}$, *then drawing a sample* u' *uniformly from the reachable neighborhood* $\mathcal{R}_\ell(u)$ *of a point* u *chosen with probability*:

$$P(u) = \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(V))} \tag{16}$$

yields a sample distribution that is an unbiased approximation of a uniform distribution over $\mathcal{R}_\ell(V)$.

Proof. The integral definition of a uniform probability distribution over $\mathcal{X}$ is

$$\mu(\mathcal{X}) \int_{\mathcal{G}} p(x)\mu(dx) = \mu(\mathcal{G}) \quad \text{forall} \mathcal{G} \subset \mathcal{X} \tag{17}$$

which can only be true if the distribution is constant and integrates to 1 over $\mathcal{X}$.

Following the same logic, we can examine the following integral:

$$c(\mathcal{G}) = \mu(\mathcal{R}_\ell(V)) \int_{\mathcal{G}} p(u')\mu(du') \tag{18}$$

$$c(\mathcal{G}) = \mu(\mathcal{R}_\ell(V)) \int_{\mathcal{G}} \sum_{u \in N(u')} \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(V))\mu(\mathcal{R}_\ell(u))} \mu(du') \tag{19}$$

$$c(\mathcal{G}) = \int_{\mathcal{G}} \sum_{u \in N(u')} \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(u))} \mu(du') \tag{20}$$

If we look at $\mathcal{G} \subset \mathcal{R}_\ell(V)$ with $N = \{u \in V | u \in \mathcal{G}\} \subset V$, the integral $c(\mathcal{G})$ reduces to

$$\begin{aligned} c(\mathcal{G}) = \mu(\mathcal{R}_\ell(N)) &- \sum_{u \in N} \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(u))} \mu(\mathcal{R}_\ell(u) \setminus \mathcal{G}) \\ &+ \sum_{u \in V \setminus N} \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(u))} \mu(\mathcal{G} \cap \mathcal{R}_\ell(u)) \end{aligned} \tag{21}$$

By defining the sets $I = \{u \in N \mid \mathcal{R}_\ell(u) \setminus \mathcal{G} \neq \emptyset\}$ and $O = \{u \in V \setminus N \mid \mathcal{R}_\ell(u) \cap \mathcal{G} \neq \emptyset\}$, the bounds on $c(\mathcal{G})$ can be expressed as

$$\mu(\mathcal{G}) - \mu(\mathcal{R}_\ell(I)) \leq c(\mathcal{G}) \leq \mu(\mathcal{G}) + \mu(\mathcal{R}_\ell(O)) \tag{22}$$

which are expected to be conservative bounds. More importantly, the true integral always lies within a limited interval around the uniform value, with both the upper and lower margins being of equal volume on average, meaning that the adopted sampling strategy approximates uniform sampling and is unbiased.

The significance of Lemma 2 should be clear as it presents the proposed sampling strategy as a viable alternative to uniform sampling in $\mathcal{R}_\ell(V)$, which is difficult in practice (and for analysis). The local deviations of the integral probabilities merely imply that the distribution of samples deviates from a uniform distribution symmetrically within a reachable band around the boundary of $\mathcal{G}$, but is not biased by the density of the existing points in V inside or around $\mathcal{G}$. It is also worth noting that for most choices of $\mathcal{G}$ the deviations are limited to a small fraction of $\mu(\mathcal{G})$. Lemma 2 allows us to prove the lemma that follows, which is key to obtaining a convergence rate and thus, probabilistic completeness.

Lemma 3. *Given a point set* V *where each point* $u \in \mathcal{F}$ *has a locally reachable region* $\mathcal{R}_\ell(u) \subset \mathcal{F}$, *and that* $\mathcal{F}$ *is* (α, β)*-expansive for some values of* $\alpha, \beta \in (0,1]$ *(Hsu, 2000), then drawing a sample* u'*uniformly from the reachable neighborhood* $\mathcal{R}_\ell(u)$ *of a point* u *chosen with probability*:

$$P(u) = \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(V))} \tag{23}$$

yields a probability of sampling u'*in the* β-LOOKOUT*of* $\mathcal{R}_\ell(V)$ *that is at least* α.

Proof. Following the proof of Lemma 2, we can choose to take the integrals c over $\mathcal{R}_\ell(V) \setminus \beta(V)$ and its complement, where $\beta(V)$ denotes the β-LOOKOUT of $\mathcal{R}_\ell(V)$, which satisfy the following:

$$c(\beta(V)) = \mu(\mathcal{R}_\ell(V)) - c(\mathcal{R}_\ell(V) \setminus \beta(V)) \tag{24}$$

$$c(\beta(V)) \geq \mu(\mathcal{R}_\ell(V)) - \mu(\mathcal{R}_\ell(V) \setminus \beta(V)) - \mu(\mathcal{R}_\ell(O)) \tag{25}$$

Downloaded from ijr.sagepub.com by guest on January 20, 2015

$$c(\beta(V)) \geq \mu(\beta(V)) - \mu(\mathcal{R}_\ell(O)) \quad (26)$$

where O is the set of points from V that lie in the β-LOOKOUT of $\mathcal{R}_\ell(V)$, which is empty, because, by definition, points in the β-LOOKOUT must be able to reach beyond $\mathcal{R}_\ell(V)$, which none of the points in V can do. Therefore, $\mu(\mathcal{R}_\ell(O)) = 0$. Furthermore, in a (α, β)-expansive space, the volume of the β-LOOKOUT is bounded below by a fraction α of the total volume, meaning that $c(\beta(V)) \geq \alpha\mu(\mathcal{R}_\ell(V))$, or, in terms of the integral probability,

$$P(u' \in \beta(V)) = \int_{\beta(V)} p(u')\mu(du') = \frac{c(\beta(V))}{\mu(\mathcal{R}_\ell(V))} \geq \alpha \quad (27)$$

which completes the proof.

For analytical purposes, we use Algorithm 1, but substitute, in place of the ChooseBestSample function, the "Voronoi-sample" strategy described in Lemma 2. Because the sampling strategy is the only relevant difference compared to the idealized EST algorithm, we are able to re-use Theorem 4.3 from Hsu (2000) in its entirety by using Lemma 3 to arrive at the same probabilistic convergence rate, under the same assumptions.

Theorem 4. *Let $g > 0$ be the volume of the goal region in $\mathcal{X}$ and γ be a constant in* (0,1]. *A sequence V of n nodes generated by Algorithm 1 with the sample u_{exp} chosen by the strategy described in Lemma 3 contains a node in the goal region with probability at least $1 - \gamma$, if $n \geq (k/\alpha)\ln(2k/\gamma) + (2/g)\ln(2/\gamma)$, where $k = (1/\beta)\ln(2/g)$.*

Proof. The proof for this theorem is identical to the proof to Theorem 4.3 of Hsu (2000) since Lemma 3 establishes that the probability of sampling a point in the lookout of $\mathcal{R}_\ell(V)$ is at least α as is required by the ideal sampler in that proof.

Theorem 4 is a significant result, even beyond the algorithms proposed in this paper. The sampling strategy adopted in the analysis can be related to many existing sampling strategies and density metrics. Most notably, the RRT algorithms implicitly generate samples in regions where the Voronoi cells are the largest by volume and, therefore, the bound on the convergence probability in Theorem 4 applies to the RRT-style algorithms as well, which may prove more convenient than existing bounds (LaValle and Kuffner, 2001). Then, many algorithms based on expansion or random walks, such as kinodynamic motion planners, rely on density heuristics (Hsu et al., 1999), and Lemma 2 implies that the sampling strategy adopted here should be the target value that density heuristics should approximate or be compared with.

2.4.2 Sampling heuristic. So far, we have ignored the presence of the heuristic function that prioritizes the search. We now look at the effect it has on the convergence of the algorithm, but first, we must characterize the difficulty of the problem with respect to the heuristic and the sought solution path.

Definition 4 (η-Entanglement). Given a free metric space $(\mathcal{F}, d_\sigma) \subset (\mathcal{X}, d)$ with connected components $\mathcal{F}_1, \ldots, \mathcal{F}_k$, and some constant value $\eta \geq 1$, then the free-space is said to be *η-entangled* if, for all $i \in [1,k]$, the total length of the collision-free path between any pair of points in $\mathcal{F}_i$ is no more than an η multiple of the "bird-flight" distance between that pair of points, i.e.

$$d_\sigma(u, v) \leq \eta d(u, v) \quad \text{for all } u, v \in \mathcal{F}_i \quad (28)$$

where $d_\sigma(u,v)$ is the length of the shortest non-colliding path between u and v, and $d()$ denotes the "bird-flight" distance between two points in $\mathcal{X}$.

Definition 4 characterizes the space in terms of how much more than a bird-flight distance one has to travel to get from any point to any other within a single connected component of the free space. Naturally, one could trivially refine the definition to a single connected component $\mathcal{F}'$ by saying that $\mathcal{F}'$ is η-entangled, or to a single-query problem by considering only a single pair of points. The key here is that the η value gives a bound on how difficult the problem can be compared with a trivial straight-line solution.

Corollary 5. *Given a pair of points $(u_{init}, u_{goal}) \in \mathcal{F}'$ where $\mathcal{F}'$ is an η-entangled connected component of $\mathcal{F} \subset \mathcal{X}$ for some finite value $\eta \geq 1$, let γ be a constant in* (0,1]. *If we have a sequence V of n nodes generated by Algorithm 1 with the following modified expansion probability*:

$$P'(u) = \frac{1}{f(u)} \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(V))} \quad (29)$$

where $f(u) \equiv d_\sigma(u_{init}, u) + d(u, u_{goal})$, then V will contain a node in the goal region $\mathcal{R}_\ell(u_{goal})$ with probability at least $1 - \gamma$, if $n \geq (k\eta/\alpha)\ln(2k/\gamma) + (2/g)\ln(2/\gamma)$, where $k = (1/\beta)\ln(2/g)$

Proof. By definition of η-entanglement, the heuristic $f(u)$ is bounded as

$$f(u) \leq \eta d_{max}, \quad d_{max} \equiv \max_{(u,v)\in\mathcal{F}'^2} d(u, v) \quad (30)$$

where the maximum distance value d_{max} is a constant for a given connected component $\mathcal{F}'$. And because d_{max} is constant, we can use it to multiply $P'(u)$ without affecting the distribution, giving

$$P''(u) = P'(u)d_{max} = \frac{d_{max}}{f(u)} \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(V))} \geq \frac{1}{\eta} \frac{\mu(\mathcal{V}_\ell(u))}{\mu(\mathcal{R}_\ell(V))} \quad (31)$$

which implies, from Lemma 3, that for a sample u', we have a bounded probability that it lies in the lookout of $\mathcal{R}_\ell(V)$:

$$P(u' \in \beta(V)) \geq \frac{\alpha}{\eta} \quad (32)$$

and from this point on, the probability bound α/η replaces the probability α in Theorem 4, reaching the probability bound stated, and completing the proof.

Downloaded from ijr.sagepub.com by guest on January 20, 2015

The value of $f(u)$ distorts the sampling probability distribution in a fashion that is concentric around the straight-line path from u_{init} to u_{goal} which will lower the worst-case probability of sampling in the β-LOOKOUT of $\mathcal{R}_\ell(V)$ if the lookout region is far from that central axis. Nevertheless, as Corollary 5 states, the entanglement of the space or the problem worsens the probabilistic convergence in a (α, β)-expansive space by a factor $1/\eta$ on the probability α.

At this point, a salient question might arise: if the heuristic worsens the convergence rate, why use it? To that, we must remind the reader that the heuristic may worsen the worst-case convergence rate, but will increase the average convergence rate in problems with low entanglement. A formal analysis of the average case is beyond the scope of this article as it would require far more elaborate constructs, but practical experience and formal analysis on discrete search algorithms, such as A*, support this conjecture.

2.4.3 Practical implications. As we emerge from the formal analysis, it is important to outline some of the key practical implications of the analysis and how they carry over to a practical algorithm.

First, the sampling strategy adopted in the analysis relies on the volume of local Voronoi regions around the points of the motion-graph. Clearly, computing those volumes is prohibitively hard and cannot be used in a practical algorithm. However, it is clear that the probability $P(NEW, FREE|u)$ is a direct analog of those Voronoi volumes. Therefore, to achieve an unbiased sampling within $\mathcal{R}_\ell(V)$, the objective should be to approximate either the volume of the local Voronoi regions or, equivalently, the probability $P(NEW, FREE|u)$. Intuition also seems to indicate that if we wish to increase the convergence rate, we should use a sampling strategy that is even more biased towards the larger Voronoi volumes, or least explored regions, i.e. inducing a *Voronoi bias*, as is the hallmark of the RRT sampling strategy.

Furthermore, the sampling strategy of Equation (12) is clearly not identical to the probabilistic sampling adopted in Corollary 5. However, given that each expansion will decrease the $P(NEW, FREE|u)$ value of its neighbors, there is a natural convection occurring in the ranking of the nodes by their expected value $r(u)$ which provides a similar probabilistic effect at a much cheaper computational cost. And as was previously noted, the introduction of further bias towards the larger local Voronoi volumes is only expected to improve the convergence by increasing the chance of sampling in the lookout of $\mathcal{R}_\ell(V)$, and thus, adopting Equation (12) is likely to be beneficial, rather than detrimental as compared to the idealized sampling strategy.

Finally, Corollary 5 assumes, in the definition of the value function $f(u)$, that the optimal path distance $d_\sigma(u_{init}, u)$ is available. Computing this value is as hard as solving the single-query problem itself and, therefore, only an approximation can be used in practice. An approximation of this value can be accumulated in the nodes of the motion graph in the same way as in discrete search algorithms. Evidently, this approximation will necessarily over-estimate the actual optimal path distance, and an asymptotic approach of the actual value hinges on the fact that, as the motion-graph grows, its edges can capture the optimal path. Therefore, adopting a connection strategy that will capture the optimal path is critical, but, fortunately, such proven connection strategies have already been developed as part of the RRG and RRT* algorithms (Karaman and Frazzoli, 2011).

In that vein, observing the proofs of asymptotic optimality for RRG/RRT* (Karaman and Frazzoli, 2011), there are clear parallels between the sequences of connected balls and the linking sequences used to prove probabilistic completeness both here and for EST-like algorithms (Hsu, 2000). Moreover, those proofs assume a uniform sampling in the reachable free-space, which we establish in Section 2.4.1 and modify with a bias in Section 2.4.2 with a bounded distortion on the sampling. Together, these observations make a strong argument to support the proposition that the idealized sampling-based A* algorithm is also asymptotically optimal when it employs the optimizing connection strategy of either the RRG or RRT* algorithms.

3. Practical sampling-based A* algorithms

In this section, we present a complete and detailed description of practical SBA* algorithms, moving away from the general and idealized discussions of the previous section. First, we present a predictive model that can approximate the node values $r(u)$ in a convenient and computationally efficient way. Second, we define a basic, concrete form of the SBA* algorithm. Finally, we present a number of important improvements to this basic algorithm.

3.1 Predictive model for node values

In the rich field of information theory, there are many tools that can help us construct a predictive statistical model for the expected value $r(u)$, but probably the most widely used is the *Kullback–Leibler* (KL) *divergence* (Kullback and Leibler, 1951), which is especially useful here due to its straight-forward relationship to the concept of *surprisal*. Given a sample drawn from the configuration-space neighborhood of node u, we want to know what is the probability that the sample is surprising (i.e. NEW) with respect to the current set of nodes in the neighborhood of u.

To represent the configuration-space neighborhood $\mathcal{N}(u)$, we use $S_u(x)$ as a probability distribution reflecting the sampling region, i.e. $S_u(x)$ gives the probability of the configuration x in the neighborhood of u. Let N_u be the set of neighboring samples within the reachable region of u, that are currently part of the motion graph or that were once attempted to be added to it (and failed due to a collision or to the connection strategy). Then, we can define $S_{N,u}(x)$ as the probability distribution reflecting the sampling of nodes

Downloaded from ijr.sagepub.com by guest on January 20, 2015

from the sampling regions around the nodes in N_u, instead of the neighborhood of u directly. This allows us to formulate the probability of a surprisal around node u as

$$P(\text{NEW}|u) = 1 - e^{-D_{KL}(S_u||S_{N,u})} \tag{33}$$

where $D_{KL}(S_u||S_{N,u})$ is the KL divergence if $S_{N,u}$ is used to represent the sampling region S_u and, thus, $e^{-D_{KL}(S_u||S_{N,u})}$ is the probability that a sample from S_u is unsurprising to the distribution $S_{N,u}$ (i.e. the sample is in a region covered by $S_{N,u}$).

Similarly, if we define C_u as the set of attempted nodes around u that have been tested and found not to be connectible to u due to a collision (or other limitations in the configuration space), then we get a sampling region representing the non-free configuration space neighborhood of u that we denote as $S_{C,u}$. Then, we get the probability of sampling a reachable node as

$$P(\text{FREE}\,|\,u) = 1 - e^{-D_{KL}(S_u||S_{C,u})} \tag{34}$$

which, together with $P(\text{NEW}\ |u)$, provide the theoretical bedrock for constructing a predictive model for the expected value $r(u)$.

Proceeding forward is a simple matter of finding a convenient and sufficiently accurate method to approximate the KL divergence between u and its neighbors. Clearly, this method can depend largely on the application domain, and can vary in sophistication, so, we proceed by demonstrating one particular choice: a method based on a Gaussian distribution for S_u and Gaussian mixtures for $S_{N,u}$ and $S_{C,u}$.

The advantages of Gaussian distributions include their applicability to any metric space, their relationship to many other types of distributions as first-order approximations of them, and the availability of a simple closed-form expression for the KL divergence. This makes the Gaussian distributions suitable to many applications, even when a Gaussian distribution is not used for generating random samples around a given node. In very general terms, a normal distribution, around node u, can be formulated as the following probability density function (pdf):

$$S_u(x) = \frac{1}{\left(2\pi\sigma_u^2\right)^{\frac{k}{2}}} e^{-\frac{\text{dist}(x,p(u))^2}{2\sigma_u^2}} \tag{35}$$

where $p(u)$ is the configuration associated with node u, σ_u is the standard deviation around u with respect to the distance metric dist $(\cdot,\cdot)$, and k is the dimensionality of the configuration space. With this definition, we can proceed and state the expression for the KL divergence between two Gaussian distributions:

$$D_{KL}(S_u||S_v) = \frac{k}{2}\left(\frac{\sigma_u^2}{\sigma_v^2} - 1 - \log\left(\frac{\sigma_u^2}{\sigma_v^2}\right)\right) + \frac{\text{dist}(p(u),p(v))^2}{2\sigma_v^2} \tag{36}$$

If we use the same standard deviation σ (defining the size of the sampling region), the expression further reduces to the following:

$$D_{KL}(S_u||S_v) = \frac{\text{dist}(p(u),p(v))^2}{2\sigma^2} \tag{37}$$

which is only part of the work, since we are really interested in $D_{KL}(S_u||S_{N,u})$.

We can model the neighborhood distribution $S_{N,u}$ as a mixture of the Gaussian distributions around each of the nodes in N_u. There is no closed-form expression for the KL divergence between Gaussian mixtures (or between a Gaussian and a Gaussian mixture), however, there are a number of suitable closed-form approximations. For example, a good approximation of the KL divergence of a Gaussian distribution and a mixture of Gaussians is the following variational approximation (Hershey and Olsen, 2007):

$$D_{KL}(S_u||S_{N,u}) = -\log\left(\sum_{v\in N_u} w_{u,v}\, e^{-D_{KL}(S_u||S_v)}\right) \tag{38}$$

where $w_{u,v}$ are the multinomial distribution factors that weight the different Gaussians of the mixture $S_{N,u}$, i.e. we must have $\sum_{v\in N_u} w_{u,v} = 1$. The above can be substituted into Equation (33) to obtain this approximation of the expected surprisal around node u:

$$P(\text{NEW}\,|\,u) = 1 - \sum_{v\in N_u} w_{u,v}\, e^{-D_{KL}(S_u||S_v)} \tag{39}$$

At this point, we need to determine appropriate values for the weights $w_{u,v}$. One obvious candidate is a uniform distribution, i.e. all weights are equal with value $w_{u,v} = \frac{1}{|N_u|}$, which effectively makes the expected surprisal the average surprisal in the neighborhood N_u. However, the average surprisal is in contradiction with the premise of the KL divergence, that is, as a measure of how accurate it would be to model the distribution S_u with the mixture $S_{N,u}$. Clearly, if we want to maximize the accuracy, we would prefer to sample from the Gaussians in $S_{N,u}$ that are more probable in the distribution S_u. This naturally leads to setting the weights to be the mean probability in S_u, i.e. we can use $w_{u,v} = S_u(p(v))$, leading to

$$P(\text{NEW}\,|\,u) = 1 - \sum_{v\in N_u} S_u(p(v))\, e^{-D_{KL}(S_u||S_v)} \tag{40}$$

which also has the advantage, like with the average value, that this surprisal value can be incrementally accumulated in every node without needing to re-examine the neighborhood N_u every time a neighbor is added to it. If we define the *density* value $d(u)$ as follows:

$$d(u) \equiv \sum_{v\in N_u} S_u(p(v))\, e^{-D_{KL}(S_u||S_v)} \tag{41}$$

Downloaded from ijr.sagepub.com by guest on January 20, 2015

and assuming that during iteration k, we add a new node v in the neighborhood of u, we can accumulate the density value as

$$d_{k+1}(u) = (1 - S_u(p(v)))\, d_k(u) + S_u(p(v))\, e^{-D_{KL}(S_u||S_v)} \tag{42}$$

where $d_k(u)$ is the density value of u *before* the addition of node v, and $d_{k+1}(u)$ is the density value after adding node v. This incremental calculation is extremely convenient from a computational point of view since traversals of neighborhoods of nodes in a graph are relatively expensive operations, even with state-of-the-art cache-optimized data structures, and minimizing their occurrences in the iterations of an algorithm can be of great benefit.

We can proceed in a similar fashion to obtain the probability of sampling an unreachable node in the neighborhood of u. This time, we define the *constriction* value $c(u)$ as follows:

$$c(u) \equiv \sum_{v \in C_u} S_u(p(v))\, e^{-D_{KL}(S_u||Sc_v)} \tag{43}$$

where we have another distribution, denoted as Sc_v, which defines the region around v where we expect nodes to be unreachable. For example, if we tried to steer from node u to node v and a collision (or other limit) was detected halfway, then we can approximate the unreachable region as being centered around v and extending to a radius equal to half the distance between u and v, which we can approximate with a Gaussian distribution to keep things consistent. Since the distributions S_u and Sc_v are different in their standard deviation, we must use Equation (36) to calculate their KL divergence. Finally, as before, the constriction value can be incrementally computed as new unreachable nodes are found:

$$c_{k+1}(u) = (1 - S_u(p(v)))\, c_k(u) + S_u(p(v))\, e^{-D_{KL}(S_u||Sc_v)} \tag{44}$$

At this point, we can come back to the expression for the expected value of a sample from the configuration space neighborhood of u, which we defined as $r(u)$ and compute using Equation (10). We get the following expression for $r(u)$,

$$r(u) = \frac{g(u) + h(u)}{(1 - c(u))(1 - d(u))} \tag{45}$$

which depends on four values associated to each node: the accumulated travel distance or cost $g(u)$, the heuristic distance or cost $h(u)$, the constriction value $c(u)$, and the density value $d(u)$.

3.2 Sampling-based A*

With a key value $r(u)$ that appropriately reflects the expected path discovery around a given vertex u, we can drive a sampling-based algorithm using the A* search strategy, and this algorithm is presented in this section. In its essence, the algorithm breaks down to two main components: the search loop and the connection strategy. The former is presented in Algorithm 2 as SBA*-Loop, while the latter is presented in Algorithms 3 and 4 as

Algorithm 2. The sampling-based A* algorithm Loop.

1: **function** SBA*-Loop Q, V, E
Require: Q is a priority queue ordered by minimum $r[u]$ value.
Require: V is the list of all vertices of the motion-graph.
Require: E is the list of all undirected edges of the motion-graph.
Require: Each vertex u has associated values for position $p[u]$, heuristic $h[u]$, accumulated distance $g[u]$, density $d[u]$, constriction $c[u]$, key value $r[u]$ and predecessor $pred[u]$.
Ensure: There are no more useful vertices to explore, or the termination condition was met.
Ensure: The $pred[u]$ values trace out the optimal path from any vertex back to the start.
2: **repeat**
3: $u \leftarrow$ Top(Q)
4: $\{p_{new}, success\} \leftarrow$ RandomWalk ($p[u]$) ▷ Generate reachable sample near $p[u]$
5: **if** *success* **then**
6: $\{P,S\} \leftarrow$ NearestNeighbors (p_{new}, V) ▷ Get set of neighbors of p_{new}
7: $v \leftarrow$ NewVertex (p_{new}, V) ▷ Create new vertex at position p_{new}
8: ConnectPredecessors (v, P, Q, V, E)
9: ConnectSuccessors (v, S, Q, V, E)
10: **else**
11: Update $c[u]$ and $d[u]$ using Equations (41) and (43). ▷ or, Record the failure
12: Requeue (u, Q)
13: **end if**
14: **until** Empty$Q$ **or** ShouldTerminate (V, E)
15: **end function**

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Algorithm 3. The sampling-based A* predecessor connection function.

```
1:  function ConnectPredecessors (u, P, Q, V, E)
Require: Q, V, and E are as defined in SBA*-Loop.
Require: P is the set of potential predecessors of u in the motion graph.
Ensure: The predecessors of u are connected, and P contains only viable predecessors.
2:    for all v∈P do
3:      {success, σ_new}← Steer(p[v], p[u])          ◁ Attempt non-colliding travel
4:      if success then
5:        Update d[u] and d[v] using Equation (41).
6:        e← NewEdge (v, u, σ_new, E)                ◁ Create new edge with travel σ_new
7:        if g[u] > g[v] + cost (σ_new) then         ◁ Choose predecessor
8:          pred[u] ←v; g[u] ←g[v] + cost (σ_new)
9:        end if
10:     else
11:       Update d[u], c[u], d[v] and c[v] using Equations (41) and (43).
12:       P←P \{v}
13:     end if
14:     Requeue (v, Q)
15:   end for
16:   Requeue (u, Q)
17: end function
```

Algorithm 4. The sampling-based A* successor connection function.

```
1:  function ConnectSuccessors (u, S, Q, V, E)
Require: Q, V, and E are as defined in SBA*-Loop.
Require: S is the set of potential successors of u in the motion graph.
Ensure: The successors of u are connected, and S contains only viable successors.
2:    I←{}
3:    for all v ∈ S do
4:      {success, σ_new}← Steer (p[u], p[v])         ◁ Attempt non-colliding travel
5:      if success then
6:        Update d[u] and d[v] using Equation (41).
7:        e← NewEdge (u, v, σ_new, E)                ◁ Create new edge with travel σ_new
8:        if g[v] > g[u] + cost (σ_new) then         ◁ Rewire if necessary
9:          pred[v] ←u; g[v] ←g[u] + cost (σ_new)
10:         I←I ∪ {v}                                ◁v has inconsistent successors
11:       end if
12:     else
13:       Update d[u], c[u], d[v] and c[v] using Equations (41) and (43).
14:       S←S \{v}
15:     end if
16:     Requeue (v, Q)
17:   end for
18:   Requeue (u, Q)
19:   repeat                                         ◁ Propagate changes in successors
20:     u← Pop (I)
21:     for all v|pred[v] = u do                     ◁ Iterate through successors of u
22:       g[v] ←g[u] + cost (σ(p[u],p[v]))
23:       I←I∪{v}
24:       Requeue (v, Q)
25:     end for
26:   until Empty (I)
27: end function
```

ConnectPredecessors and ConnectSuccessors, respectively. In both pseudo-code presentations, many of the implementation details have been omitted for brevity, outlining only the main logic of the algorithm.

3.2.1 Search and connection strategies. The main loop simply keeps a priority queue for all of the nodes in the motion-graph with respect to their associated key value $r[u]$,[1] prioritizing the nodes with minimum key values. At

Downloaded from ijr.sagepub.com by guest on January 20, 2015

each iteration, the best node is obtained from the queue and a sample is drawn from its neighborhood with an attempt to achieve a collision-free path to that sample: this process is performed by an application-specific RandomWalk function. If the sampling and connection were unsuccessful, the failure must be recorded in the cumulative values of the density $d[u]$ and constriction $c[u]$ using Equations (41) and (43), respectively, or using a recursive formula if possible, as per the remarks in the previous section. Then, the node's rank in the priority queue is updated since its key value was changed through the recorded failure. If the sampling and connection were successful, then the new sample can be inserted into the motion graph by first obtaining sets of neighbors (predecessors and successors), then creating a new vertex in the motion graph and finally, calling Algorithms 3 and 4 to create the connections. The sets of nearest neighbors can have a size (number of neighbors) and range that are either fixed or dictated by an adaptive strategy such as the so-called *star neighborhood* (Karaman and Frazzoli, 2011).

The loop finishes when there are no more nodes in the queue or when some specific termination condition is reached (checked with the ShouldTerminate function). In non-optimizing sampling-based motion planners, the natural termination condition is reached when a connection is established between the start and goal locations. However, like most asymptotically optimal planners, the SBA* algorithm does not have a natural stopping criterion. In this case, this is manifested by the queue never becoming empty as samples are consistently requeued to it. One option to cause a natural termination is to impose a threshold on the density $d[u]$ or value $r[u]$ above which the samples are no longer requeued, which will eventually exhaust all valuable samples and empty the queue. This termination method introduces additional parameters to tune and it is unclear what repercussions it would have on the convergence of the algorithm. However, simple termination conditions that are often used in sampling-based algorithms can be easily applied here, such as terminating after a certain number of iterations have passed or when a sufficiently good solution was obtained.

In this nominal version of the sampling-based A* algorithm, an exhaustive and eager connection strategy is used, as seen in Algorithms 3 and 4, which is similar to the connection strategy in the RRG algorithm (Karaman and Frazzoli, 2011); a more economical alternative will be presented in Section 3.3.1. The ConnectPredecessors routine proceeds to connect a new vertex u to its potential predecessors, and, at the same time, it finds the optimal predecessor in that set. Then, the ConnectSuccessors routine makes the successor connections going from the vertex u to other neighboring vertices in the motion graph. Both routines rely on a Steer function that attempts to steer between two points and returns a record of the path σ_{new} that links the two points with cost cost (σ_{new}), if successful. In addition, of course, the successors must be recursively traversed to update their accumulated distance $g[v]$ and be re-positioned in the priority queue. The main difference between Algorithms 3 and 4 as compared with the RRG connection routines is the work done to update the density, constriction, and key values, as well as keeping the priority queue consistent with those values.

3.2.2 Implementation remarks. At this point, we make a few general implementation remarks: these will be obvious to any seasoned implementer, but worth mentioning nevertheless. First, our presentation of the algorithms include explicit mentions of the points at which the priority queue is updated, which is not customary in standard pseudo-code expositions. We aim to show explicitly where those updates are needed since they can represent an important computational cost in an implementation.

Second, the recursive updates of the accumulated distances, after re-wirings (or successor connections) have been performed, are presented as a non-recursive breadth-first traversal (via the inconsistent set I). Here, we want to make it clear that an implementation through actual recursive calls is out of the question in this type of application given the potential great depth of any branch of the optimal motion tree.

Third, when the local planner and distance metric are asymmetric, which is quite common in practice, the set of potential nearest predecessors to a given configuration point is different from the set of potential nearest successors from that point. The nearest-neighbor query for a set of predecessors and successors can, in general, be performed in one operation, thereby making fairly substantial savings in computational effort, especially in terms of making good use of cached memory. On the contrary, if the planner and corresponding distance metric are symmetric, i.e. the path and distance from A to B are exactly the same as the path and distance from B to A, then the algorithm must simply use the same set of neighbors for predecessors and successors. Furthermore, in that case, Algorithms 3 and 4 can be combined into a single routine, allowing some beneficial modifications such as the removal of the second set of steering attempts needed when connecting successors.

Finally, as in most sampling-based motion planners (and even planning on a fixed motion graph), the topology of the motion graph is essentially that of a nearest-neighbor graph, and most operations done (re-wirings) are local as well and, thus, it is recommended to choose a storage strategy that reflects this pattern in the memory layout of the nodes, i.e. nodes that are close to each other in configuration space ought to be close to each other in physical memory. There are some appropriate cache-aware or cache-oblivious data structures that can be used for that purpose (Kasheff, 2004; Chowdhury, 2007; Jamriska et al., 2012). Moreover, an important computational cost in all sampling-based motion planners is the nearest-neighbor queries performed at each iteration. It is thus important, for performance sake, to have an effective space-partitioning tree to resolve those queries in $O(\log(N))$ time (Fu et al., 2000), which often implies a

Downloaded from ijr.sagepub.com by guest on January 20, 2015

cache-friendly data structure to keep the performance from degenerating towards $O(N)$ time due to cache thrashing.

3.3 Refining the sampling-based A*

Thus far, we have presented a very general version of the sampling-based A* algorithm, but it has many shortcomings which can be rectified, especially when simplifying assumptions can be made. In this section, we present a number of improvements to the general algorithm with the aim of mitigating those problems. First, because the general version uses an RRG connection strategy, an obvious improvement is to adopt an RRT* connection strategy, i.e. pruning sub-optimal edges and evaluating collisions in a lazy fashion, that is, under the assumption that full-neighborhood connectivity is not required for the density computations. Second, one shortcoming of the SBA* algorithm is that it is biased towards improving current optimal paths (i.e. *exploitation*) and not towards finding a connection to the goal region. Given that this shortcoming is also an issue with the A* algorithm, we employ the same solution that is employed in that domain, i.e. the *Anytime A** heuristic (Likhachev et al., 2003) is used to drive the growth of the motion-graph more rapidly towards the goal. Finally, a fundamental issue with the *exploitation* bias is the lack of *exploration* of the unknown regions of the configuration space, which could potentially yield better paths. To solve this problem, we employ a *simulated annealing* (Ingber, 1996) approach that balances the SBA* algorithm with a purely *exploratory* algorithm, that is, RRT*.

3.3.1 Lazy and pruned connections. In this section, we present an alternative to the connection strategy presented in Algorithms 3 and 4 which prunes away sub-optimal edges and delays collision-checking to the point of creation of a new optimal edge. This alternative strategy is certainly more economical in terms of computational time and memory required to store the motion graph, which now becomes a motion tree due to the pruning of sub-optimal edges. However, it is important to note the assumptions that must be made and a trade-off involved in choosing this strategy.

The first necessary assumption is that the distance metric must reflect the actual cost of travel between two nodes (if a collision-free path exists); this assumption is necessary to use the lazy collision checking strategy. In the general SBA* algorithm, a steering routine is always called before the distance value is required, i.e. *eagerly* and, thus, the general form should be used when a simple and relatively inexpensive distance metric cannot accurately reflect the steering cost-to-go.

It is also necessary to assume that recursive (or incremental) formulations for the density and constriction values are available for the given problem. If we prune sub-optimal edges from the motion-graph, then it implies that nodes will only be connected to their "optimal neighbors", which means that re-computing the density or constriction values by surveying the neighborhood will not accurately reflect the true neighborhood of the node, unless a nearest-neighbor search is conducted, which would be prohibitively expensive relative to keeping a fully connected motion graph. Hence, we recommend the general form if a survey of the neighborhood is required to update the density or constriction values associated with the nodes.

Finally, there is a trade-off involved in checking collisions lazily in the SBA* algorithm. Because the SBA* heuristic uses a constriction value to reflect the likelihood of sampling colliding points in the neighborhood of a point, if collision is only checked for potentially optimal edges of the motion graph, then the constriction will be underestimated in general since it will not capture colliding paths in sub-optimal directions. Fortunately, this does not seem to have a significant impact on the algorithm, and in fact, the SBA* algorithm can still work even without a constriction value. In our opinion, this trade-off is reasonable given the potential benefit that lazy collision checking can have on overall performance.

Mostly for the sake of completeness, we present the connection strategy in the form of Algorithms 5 and 6. These constitute rather straight-forward modifications to the general connection algorithms. Mainly, the distance metric is first computed and tested for giving rise to an optimal edge and, if so, non-colliding steering is attempted and, if successful, a new edge is created to replace the existing incident edge. What is noteworthy, however, are the updates to the density and constriction values. Whether a connection is useful or not (optimal), and whether a connection is possible or not (collision-free), the density value must still be updated to reflect that a new neighboring point has been added to the motion graph and, thus, the update is performed for all potential predecessors and successors. As per the aforementioned trade-off, the constriction value can only be updated once a collision-free travel was attempted and failed, as seen in Algorithms 5 and 6. Clearly, the astute reader will notice that this connection strategy is in effect the same as that used in the classic RRT* algorithm.

3.3.2 Anytime A as goal bias.* As is well known, the A* algorithm is driven to fully explore the regions around the optimal path, but not towards actually finding a feasible path as fast as possible. The SBA* algorithm will behave in a similar fashion, generating samples around the optimal regions to try and enrich those areas (up to a certain density), but there is no bias towards actually reaching the goal region.

In the domain of discrete path-planning, this problem can be solved with the so-called *Anytime A** algorithm (Likhachev et al., 2003). The idea with this method is to artificially inflate the heuristic value such that priority for choosing nodes is biased towards those nodes that have a smaller heuristic value, i.e. are closer to the goal region. This method rapidly finds a feasible solution, and as the heuristic values are deflated back to their true values, the

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Algorithm 5. The lazy sampling-based A* predecessor connection function.

```
1:  function LazyConnectPredecessors (u, P, Q, V, E)
Require: P, Q, V, and E are as defined in ConnectPredecessors.
Ensure: The optimal predecessor of u is connected.
2:    for all v ∈ P do
3:      Update d[u] and d[v] using Equation (42).
4:      if g[u] > g[v] + dist (p[v],p[u]) then
5:        {success,σ_new} ← Steer (p[v], p[u])          ◁Attempt non-colliding travel
6:        if success then
7:          RemoveEdge (pred[u], u, E)                  ◁ Prune existing sub-optimal edge.
8:          e ← NewEdge (v, u, σ_new, E)                ◁ Create new edge with travel σ_new
9:          pred[u] ← v; g[u] ← g[v] + cost (σ_new)
10:       else
11:         Update c[u] and c[v] using Equation (44).
12:       end if
13:     end if
14:     Requeue (v, Q)
15:   end for
16:   Requeue (u, Q)
17: end function
```

Algorithm 6. The lazy sampling-based A* successor connection function.

```
1:  function LazyConnectSuccessors (u, S, Q, V, E)
Require: S, Q, V, and E are as defined in ConnectSuccessors.
Ensure: The optimal successors of u are connected.
2:    I ← {}
3:    for all v ∈ S do
4:      Update d[u] and d[v] using Equation (42).
5:      if g[v] > g[u] + dist (p[u],p[v]) then
6:        {success, σ_new} ← Steer (p[u], p[v])         ◁ Attempt non-colliding travel
7:        if success then
8:          RemoveEdge (pred[v], v, E)                  ◁ Prune existing sub-optimal edge.
9:          e ← NewEdge (u, v, σ_new, E)                ◁ Create new edge with travel σ_new
10:         pred[v] ← u; g[v] ← g[u] + cost (σ_new)
11:         I ← I ∪ {v}                                 ◁ v has inconsistent successors
12:       else
13:         Update c[u] and c[v] using Equation (44).
14:       end if
15:     end if
16:     Requeue (v, Q)
17:   end for
18:   Requeue (u, Q)
19:   repeat                                            ◁ Propagate changes in successors
20:     u ← Pop (I)
21:     for all v|pred[v] = u do                        ◁ Iterate through successors of u
22:       g[v] ← g[u] + cost (σ(p[u],p[v]))
23:       I ← I ∪ {v}
24:       Requeue (v, Q)
25:     end for
26:   until Empty(I)
27: end function
```

product of the algorithm is brought back to being the optimal path, thus, giving it an *anytime* quality which is useful in many applications.

For the SBA* algorithm, this anytime strategy could prove to be very useful as a bias towards establishing a connection to the goal region. The problem statement is as follows: we seek an optimal path that connects to the goal, that is, we are looking for a solution to the following problem:

$$\min_u (g(u) + h(u)) \quad \text{with } h(u) = 0 \tag{46}$$

which, in the classic A* heuristic, does not include the equality constraint because an exhaustive search is

Downloaded from ijr.sagepub.com by guest on January 20, 2015

guaranteed to find the node that satisfies the constraint if one exists. If we hope to find a solution that satisfies the equality constraint, without requiring an exhaustive search, we must put emphasis on satisfying the constraint, as is usually done in constrained optimization algorithms. For this purpose, we can introduce a Lagrange multiplier $\lambda \in [0, \infty[$:

$$m(u) \equiv g(u) + h(u) + \lambda h(u) \tag{47}$$

The effect of a large value for the Lagrange multiplier is to drive the search more greedily towards satisfying the constraint, i.e. establishing a connection to the goal region. Once the connection has been established, the Lagrange multiplier can be relaxed down to 0. However, given that a single connection to the goal is probably not useful enough in practice, the relaxation must be progressive.

By going through the same derivation as in Section 3.1, one can obtain the following expected value for a given node u :

$$r_{any}(u) \equiv E_{\mathcal{N}(u)}[m(u)] = \frac{g(u) + h(u)}{(1 - c(u))(1 - d(u))} + \lambda(h(u) - h_b) \tag{48}$$

where h_b represents an optional goal bias in the sampling strategy which measures how much closer to the goal a new sample is expected to be compared with node u. The above heuristic formula illustrates how inducing a constant bias towards the goal, as is often done in uni-directional sampling-based planning algorithms, may not have a significant impact on the prioritization of nodes. We also caution the reader that a naive application of a goal bias, such as repeatedly attempting to expand directly towards the goal, could have negative impacts on the entropy of the overall search. For the remainder of this paper, we will assume no bias, i.e. $h_b = 0$.

3.3.3 Balancing RRT and SBA* with simulated annealing.* The final improvement to the SBA* algorithm is aimed at counter-balancing its exploitation bias with an exploratory method. The key issue here is in generating new nodes for the motion graph that are driven towards unexplored regions of the configuration space. This is commonly referred to as the *Voronoi bias* and is the central element of the RRT family of methods (Kuffner and LaValle, 2000). In RRT methods, new nodes are generated by taking a random sample from the configuration space, finding its nearest neighbor in the current motion graph, and expanding that node towards the random sample. This method can be used as a drop-in replacement for the expansion method of the SBA* algorithm, which would essentially yield the RRG or RRT* algorithm, if using the full connection strategy or the pruned connection strategy, respectively. It thus follows that this node generation method can serve to easily introduce exploration into the SBA* algorithm.

The only problem remaining now is to strike a balance between nodes generated with the SBA* strategy and with the RRT* strategy, i.e. between *exploitation* and *exploration*. A classic technique used for this purpose in the context of general optimization problems is *simulated annealing* (Ingber, 1996). We propose to employ this strategy in the context of motion planning, specifically as a means of balancing the two sampling strategies. The choice between exploitation and exploration is determined randomly at each iteration with a probability driven by a temperature value which cools as the algorithm progresses. Initially, at high temperatures, there is a greater chance of choosing exploration, and as cooling takes effect, the focus is shifted towards exploitation.

The classic cooling formula (Ingber, 1996) used in simulated annealing methods yields the following probability of choosing an exploitation step:

$$\varepsilon_{th} = 1 - e^{\frac{-T_0}{\log(N)}} \tag{49}$$

where T_0 is the initial temperature and N is the number of iterations performed. There are, of course, other alternative formulations to drive the cooling, but the above is both theoretically optimal under certain assumptions and works well in practice.

Algorithm 7 shows the complete SBA* algorithm with simulated annealing to schedule the generation of RRT-style nodes. As one can see, it simply chooses to generate RRT nodes or SBA* nodes randomly depending on the simulated annealing schedule, and uses the same connection strategy in either case. This algorithm combines the best of both sampling-based algorithms. The RRT* algorithm has the advantage of rapidly exploring the space, but generating enough nodes to sufficiently refine the solution is often prohibitively expensive. On the other hand, the basic SBA* strategy has the opposite problem, it can leave many regions of the space unexplored, thus missing potential optimal solutions. Through the smooth transition from exploration to exploitation, we can retain the essential benefits of the RRT* algorithm while being able to refine the optimal paths more effectively.

4. Simulation results

In this section, we wish to present as complete a picture as possible to characterize the SBA* algorithm empirically. To this end, we first present a set of results on a simple two-dimensional point robot such that visual representations of the resulting motion graphs are clear. Moreover, we present statistical analysis for environments representative of the intended application and also a brief qualitative assessment of more difficult scenarios. Then, to better understand the behavior of the SBA* algorithm with respect to dimensionality, we plan paths through empty spaces (i.e. obstacle-free) of increasing dimensionality. Finally, we apply the proposed method to a real-world example with a 7-dof manipulator performing a static interception task through a

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Algorithm 7. The sampling-based A* algorithm loop with simulated annealing.

function SA-SBA*-LOOP (Q, V, E, T_0)
Require: Pre-conditions are the same as SBA*-LOOP.
Require: T_0 is the initial temperature value.
Ensure: Post-conditions are the same as SBA*-LOOP.
  repeat
    $\varepsilon_{th} \leftarrow 1 - e^{\frac{-T_0}{\log(N)}}$ ◁ Compute current entropy
    if RANDOM([0, 1]) $> \epsilon_{th}$ **then**
      $u \leftarrow$ TOP (Q) ◁ Generate SBA* node
      $\{p_{new}, success\} \leftarrow$ RANDOMWALK ($p[u]$)
    else
      $p_{rand} \leftarrow$ SAMPLE() ◁ else, Generate RRT* node
      $u \leftarrow$ NEARESTPREDECESSOR (p_{rand}, V)
      $\{p_{new}, success\} \leftarrow$ STEER ($p[u]$, p_{rand}) ◁ Attempt non-colliding travel
    end if
    if *success* **then**
      $\{P,S\} \leftarrow$ NEARESTNEIGHBORS (p_{new}, V) ◁ Get set of neighbors of p_{new}
      $v \leftarrow$ NEWVERTEX (p_{new}, V) ◁ Create new vertex at position p_{new}
      LAZYCONNECTPREDECESSORS (v, P, Q, V, E)
      LAZYCONNECTSUCCESSORS (v, S, Q, V, E)
    else
      Update $c[u]$ and $d[u]$ using Equations (41) and (43). ◁ or, Record the failure
      REQUEUE (u, Q)
    end if
  until EMPTY (Q) **or** SHOULDTERMINATE (V, E)
end function

lightly cluttered environment and analyse its performance in Monte Carlo runs.

In all cases, different variants of the proposed SBA* algorithm will be compared with each other and with the RRT* algorithm. The RRT* algorithm is one of the best performing algorithms for these types of scenarios, i.e. static environments, no kinodynamic constraints, and assuming uni-directional planning. We omit comparison with the RRT algorithm as it shares the same sampling method as RRT* and is otherwise superseded by the RRT* optimal connection strategy.

It is important to note that all of the algorithms presented were tested with the same software written by the same implementer, only changing the core algorithmic loop. The implementation is in $C++$ and is available open-source under GPLv3 as part of the *ReaK*[2] library. Nearest-neighbor queries are performed, in all cases, using a *dynamic vantage-point tree* (DVP-tree) (Fu et al., 2000) implementation which is capable of efficient queries for any distance metric (including asymmetric metrics). In all cases, the vertices of the motion graph are stored within the DVP-tree's data storage, which, in turn, is a quaternary tree laid out in breadth-first order in contiguous memory, providing better locality for efficient use of cache memory.

4.1 Two-dimensional point-robot environments

As a first set of tests, we present a number of results on a simple two-dimensional environment represented by a black-and-white image whose white pixels represent free areas and black pixels represent obstacles. This simple environment is especially useful for presentation of the shape and evolution of the motion graph as generated by the different path-planning algorithms. The comparisons mainly involve the RRT* algorithm as a reference point, the SBA* algorithm and the SBA* algorithm with simulated annealing (SA-SBA*; as per Section 3.3.3). For both SBA* variants, the lazy connection strategy and the anytime heuristics are used, as per Sections 3.3.1 and 3.3.2, respectively. In all experiments, the initial relaxation factor for the anytime heuristic was 5.0, and the initial temperature for the simulated annealing schedule was 2.0, which results in approximately 500 nodes generated by the RRT-style mechanism during the entire initial high-entropy phase of the SA-SBA* algorithm.

The distance metric used here is the Euclidean distance between points, measured in pixels, and the distance is also used as the travel cost along edges. For each sample u, with associated position $p(u) \equiv (x, y)$, the heuristic value is computed as

$$h(u) \equiv \left\| p_{goal} - p(u) \right\|_2 \tag{50}$$

where p_{goal} is the position of the goal pixel and $\|\cdot\|_2$ denotes the Euclidean norm.

4.1.1 Single runs. In this section, we present a qualitative assessment of the SBA* algorithms by presenting the motion-graphs resulting from single runs of the different variants of SBA* and RRT* on example environments. The idea is to illustrate, compare and contrast the

Downloaded from ijr.sagepub.com by guest on January 20, 2015

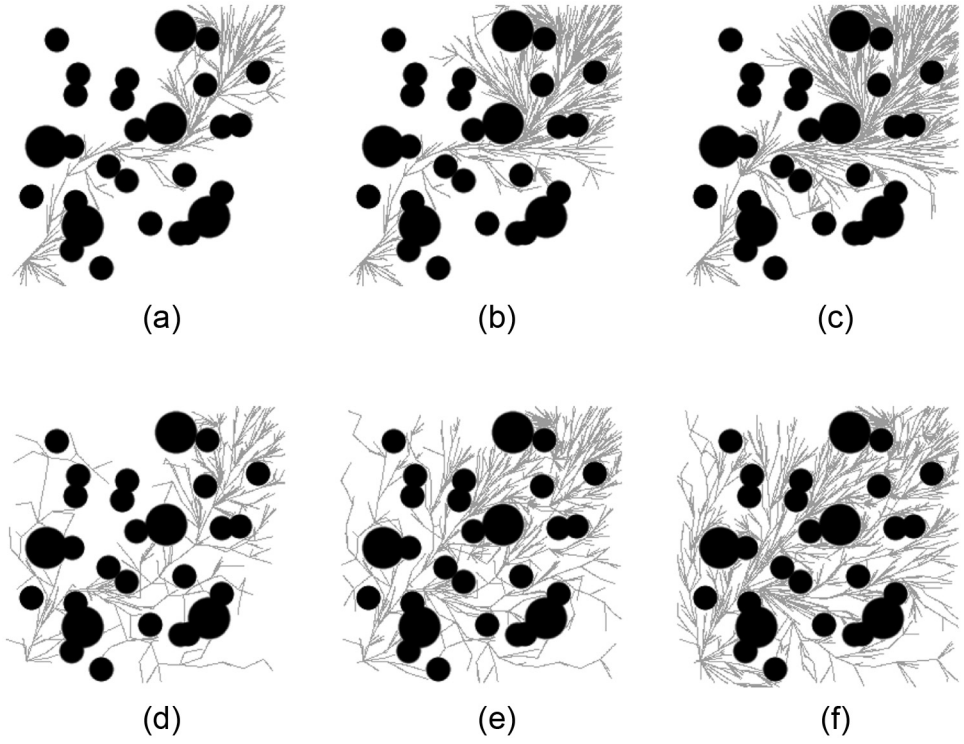


Figure 1. Motion-graphs generated on moderately cluttered environment: (a) SBA* at 500; (b) SBA* at 1000; (c) SBA* at 1500; (d) SA-SBA* at 500; (e) SA-SBA* at 1000; (f) SA-SBA* at 1500.

characteristics of the different algorithms. More quantitative assessments with Monte Carlo runs follow in the next section.

Figure 1 shows a comparison of the evolution of the SBA* algorithm versus the SA-SBA* algorithm for a moderately cluttered environment with the start position at the bottom-left corner and the goal location at the upper-right corner of the image. As one can see, the anytime SBA* algorithm evolves by pushing strongly for an expansion towards the goal region, but as the area towards the goal region gets increasingly dense, more nodes are generated back towards the starting area to obtain more uniform density of nodes all along the region around the optimal path. As mentioned earlier, this greedy push to make a connection to the goal region makes the algorithm susceptible to local minimas and variance in the results. When exploration through RRT* is scheduled with simulated annealing, one can see from the bottom row in Figure 1 that the initial growth is much more reminiscent of the RRT* algorithm, as expected, but it rapidly adopts an SBA* strategy which helps fodder the optimal path region while leaving the far-reaching regions virtually unaffected. This latter aspect is very desirable because continuing with RRT* in the hopes of optimizing the optimal path wastes a lot of computational capital on far-reaching, sub-optimal regions of the configuration space.

Continuing on the topic of using RRT* entirely, we present Figure 2 which compares the RRT* algorithm and the SA-SBA* algorithm, both with and without branch-and-bound pruning, after an equal amount of iterations (1500) on a highly cluttered environment. From Figure 2(a), one can see the aforementioned issue about continuing to generate an RRT* tree in the hopes of refining the solution, that is, the optimal path does not improve significantly since the additional computational capital is spread over the entire motion graph. A standard method to mitigate this problem is to use branch-and-bound pruning, which is the simple act of eliminating nodes from the motion graph that cannot possibly be part of a path that would be shorter than the current best solution. Figure 2(b) shows the result of 1500 iterations of RRT* with branch-and-bound pruning. In this particular run, the final motion graph contains about 400 nodes, including about 50 nodes added since the first pruning pass (after the first solution was found). This highlights a problem with RRT* and branch-and-bound: the pruning strategy wants to limit the search to the optimal region while the sampling strategy is biased towards unexplored regions. This conflict results in a lot of iterations with very little progress in the number of nodes or in total cost of the optimal path, because the overlap between the unexplored areas of the configuration space and the areas that can pass the pruning criteria is very small, leading, in this particular case, to a new node to iteration ratio of less than 10%. This is a considerable waste of effort since rejected samples must still be added to the tree before being rejected, and thus, the computational expense is per iteration, not per successfully added node.

By contrast, the SA-SBA* algorithm is much more graceful in the solution refinement phase. As seen from Figure 2(c), the overall continuation of the SA-SBA*

Downloaded from ijr.sagepub.com by guest on January 20, 2015

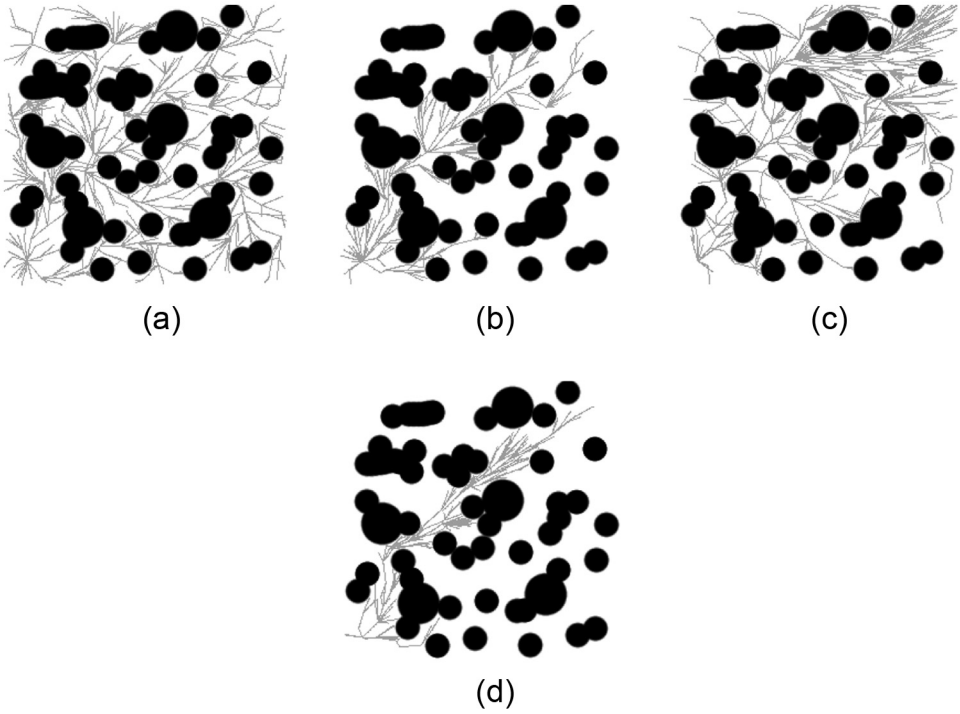


Figure 2. Motion-graphs generated after 700 iterations on a highly cluttered environment: (a) RRT*; (b) RRT* with BnB; (c) SA-SBA*; (d) SA-SBA* with BnB.

algorithm past the initial solution is much more effective at creating a richer motion-graph along the optimal path, while leaving uninteresting regions unaffected. This generally leads to more progress towards refining the current solution, and thus, an earlier termination of the overall algorithm. With branch-and-bound pruning, an additional benefit of the SA-SBA* algorithm comes to light. As seen from Figure 2(d), when branch-and-bound pruning is applied to the motion-graph, the resulting motion-graph has, in fact, about twice the amount of nodes (about 750 in this case) and is much denser around the optimal path. The key difference here is that the pruning criteria and the sampling strategy are in accord with each other, leading, in this particular case, to a new node to iteration ratio above 50%.

4.1.2 Monte Carlo runs. In this section, we compare the three main algorithms, RRT*, SBA* and SA-SBA*, with Monte Carlo runs, bearing more statistical significance. There are three variables we are interested in: the quality of the solutions, the termination rates, and the execution time. For the purpose of exposition, we have gathered the results of 100 Monte Carlo runs on the moderately cluttered environment (see Figure 1) and on the highly cluttered environment (see Figure 2) as a representative set of results. Many more tests have been performed, always showing comparable results.

We begin by assessing the quality of the solutions obtained from the different algorithms. To this end, Figures 3(a) and 3(b) show the average solutions found among 100 runs of the 3 algorithms. The total travel distance is measured in pixels of the image, with the absolute optimal solution being about 301 pixels. The main observation we can make here is that the SBA* and SA-SBA* algorithms significantly out-perform the RRT* algorithm on early solutions (below 1000 nodes). The early solutions produced by the SBA* algorithm are the best of the three methods tested, however, the reliability of the SBA* algorithm is only comparable to or slightly better than the RRT* algorithm (see Figure 4) and it is rather slow at improving the solutions in the later phase. This suggests that the exploitative (or greedy) nature of the algorithm causes it to more often miss the global optimum in favor of alternative sub-optimal paths (i.e. local minimas). On the other hand, the SA-SBA* appears to find very good early solutions, with more reliability than either the RRT* or SBA* algorithms, and exhibits an asymptotic behavior that closely matches that of the RRT* algorithm.

To further assess the reliability of the three methods, Figures 4(a) and 4(b) show the number of planners, out of 100, that have found at least one solution after a given number of iterations. What is clear from those results is that the SA-SBA* algorithm is very reliable at finding an initial solution quickly, with a median arrival at about 250 iterations and a success probability above 99% after 600 iterations for both the moderately and highly cluttered environments. By contrast, those figures are more than doubled for the RRT* algorithm. As expected, the SBA* algorithm becomes less reliable as the cluttering in the environment is increased, due to the higher likelihood of

Downloaded from ijr.sagepub.com by guest on January 20, 2015

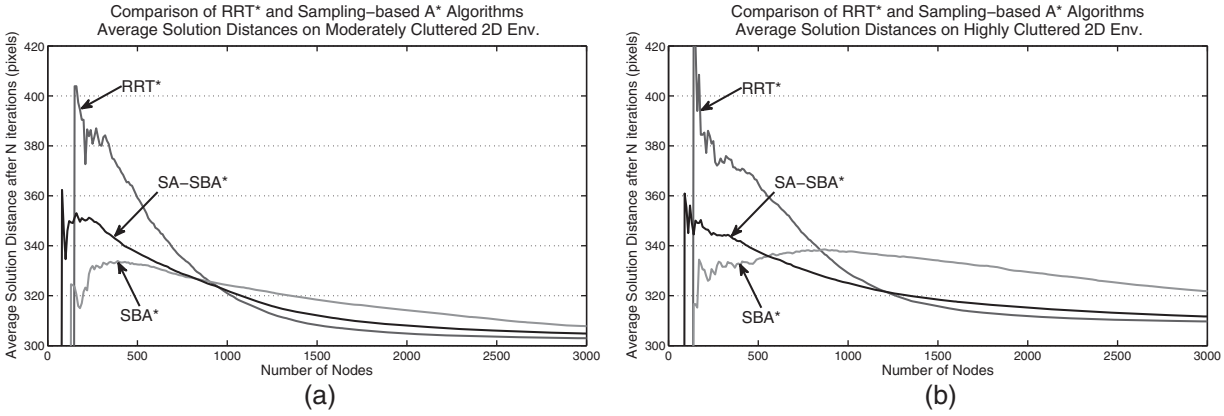


Figure 3. Monte Carlo results comparing RRT*, SBA* and SA-SBA* on (a) moderately and (b) highly cluttered environments, showing the average cost of the solutions found by a 100 planners plotted against the number of iterations.

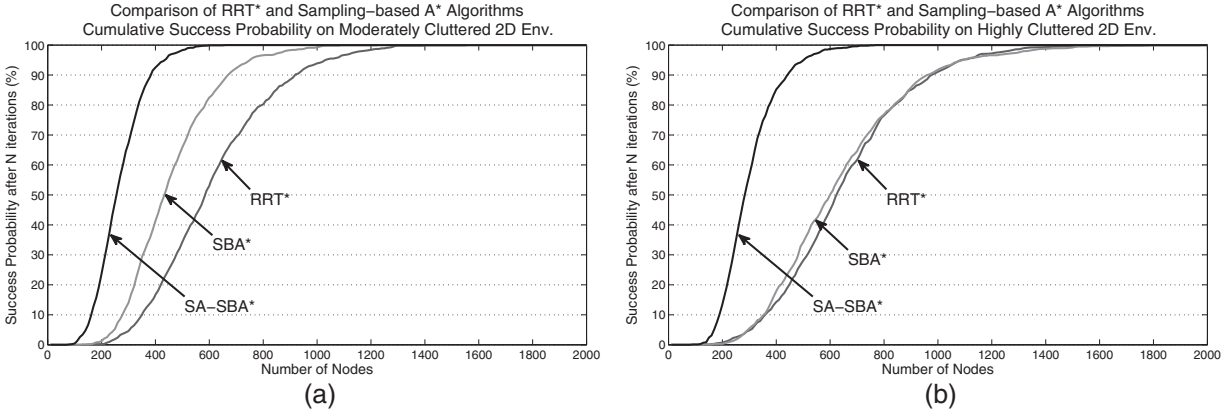


Figure 4. Monte Carlo results comparing RRT*, SBA* and SA-SBA* on (a) moderately and (b) highly cluttered environments, showing the percentage of successful planners versus the number of iterations.

being led astray into dead-ends or unfruitful local minimas. The most interesting phenomenon observed here is that the SA-SBA* algorithm is more reliable than both algorithms. One would expect that since the SA-SBA* algorithm is part RRT* and part SBA*, under a simulated annealing schedule, that it would exhibit success rates that fall between those two constituent algorithms. Clearly, this is a combination that performs better than the sum of its parts. This is, in fact, in accordance to the principles that led to the development of this novel method as explained in Section 3.3.3. By combining an exploratory method with an exploitative method, we solve the main shortcomings of both methods, leaving us with a method that performs better than either of those constituents.

The final variable we are interested in is the actual execution times of the algorithms. Figure 5 presents the average execution time for each additional iteration, measured in microseconds. The first striking observations are three spikes of increasing magnitude at exponential intervals, around the 100th, 350th and 1400th iterations, respectively. This is a classic manifestation of the amortized-constant

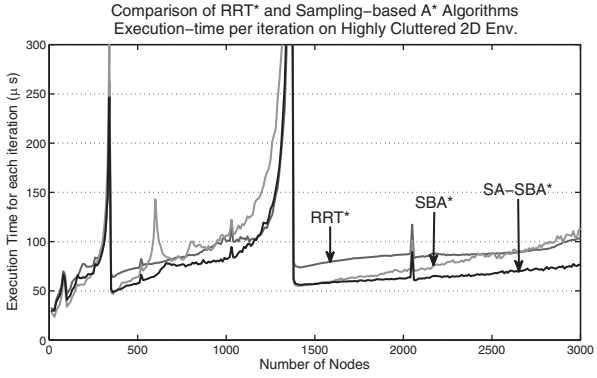


Figure 5. Monte Carlo results comparing RRT*, SBA* and SA-SBA* on the highly cluttered environment, showing the average execution time (in μs) required for each iteration plotted against the number of iterations (or number of nodes of the motion graph).

growth of the contiguous-storage container used to store the vertices of the motion graph. On the platform used for these tests, the memory capacity seems to quadruple

Downloaded from ijr.sagepub.com by guest on January 20, 2015

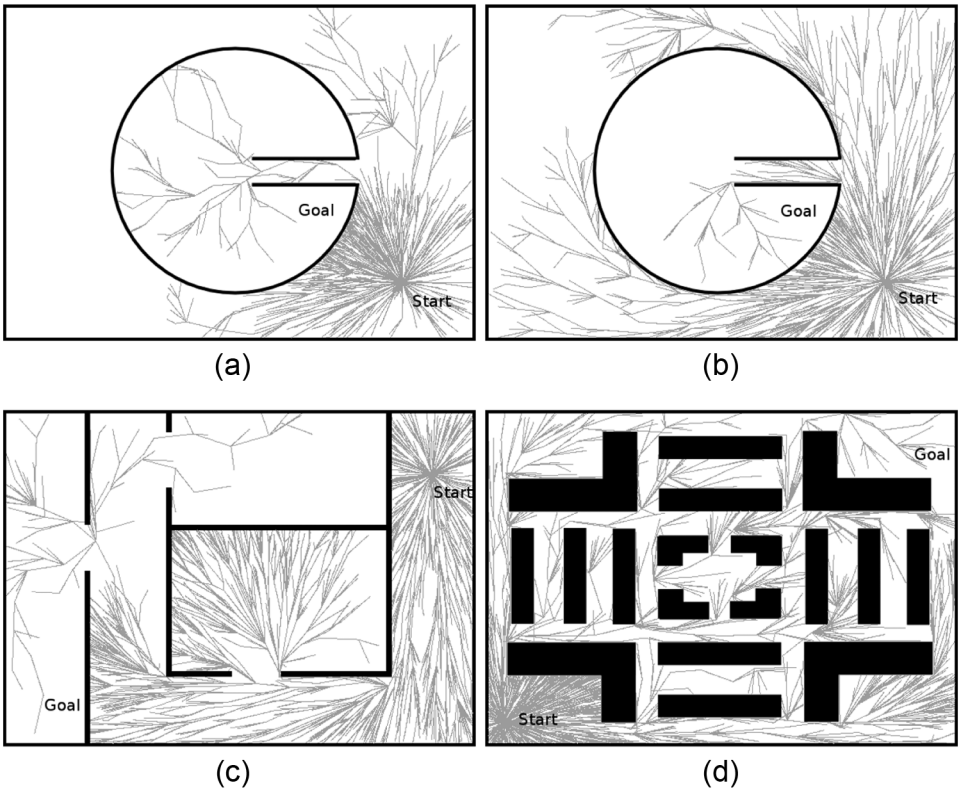


Figure 6. Motion-graphs generated on "indoor" environments such as a bug-trap, an office space, and a symmetric world, all captured at the moment when start-goal connectivity is achieved: (a) SBA* at 1600; (b) SA-SBA* at 1600; (c) SA-SBA* at 2100; (d) SA-SBA* at 3000.

whenever it is exhausted, leading to those spikes, which amortize to a constant run-time cost and are thus negligible artifacts from an analytical point of view. The second observation is that in all cases, the execution time per iteration is a logarithmic function of the number of nodes in the motion graph. This is the cost incurred by nearest-neighbor queries performed during each iteration. Thus, as expected, the empirical time complexity of each algorithm is $O(N \log N)$ (for N nodes). Finally, we observe that the computational cost of each algorithm is essentially the same, with the SBA* algorithms being marginally better, which is attributable to the fact that only one nearest-neighbor query is required for a SBA* iteration, as opposed to two for an RRT* iteration (one for the expansion and one to find the connection neighborhood).

4.1.3 Indoor environments. Although it is not the intended application, it is interesting to take a brief look at the behavior of the SBA* algorithms in "indoor" environments. By this, we mean the typical office environment, which are characterized by poor connectivity (in the sense of ϵ-goodness or (α, β)-expansiveness), narrow passages and entangled solutions (in the sense of η-entanglement). As is clear from the convergence rates derived in Theorems 4 and 5, these factors have a direct effect on the performance of the idealized SBA* algorithm. It is thus expected that the SBA* algorithms tested will be adversely affected by these difficulties as well.

Figure 6 shows a number of examples of the SBA* and SA-SBA* motion graphs on a number of these difficult spaces, such as the classic "bug-trap" environment, a simple office space, and a classic symmetric environment riddled with narrow corridors and poor connectivity. The number of nodes displayed in Figure 6 correspond to the moment when start-goal connectivity is achieved, at 1600, 2100, and 3000, for each environment. The RRT* solved the same problems with 1000, 800, and 1000 nodes, respectively. Even without statistical analysis, it is clear that the number of nodes required for a solution is quite substantial in comparison with an exploratory algorithm such as RRT*, as expected. The algorithms nevertheless make progress and succeed to solve the problems. Another qualitative observation in these scenarios is that it appears that narrow passages are less of an obstacle to the planners than the entanglement of the solution paths, which is, again, expected from the direct effect that entanglement has on the probability of progress by the expansions. By rough inspection of the problems used here, we could estimate that the η value for the entanglement of these problems range between 1.5 and 3. Overall, these results confirm the idea that the SBA* algorithms are more appropriate for problems that feature low entanglement of the solution paths.

Downloaded from ijr.sagepub.com by guest on January 20, 2015

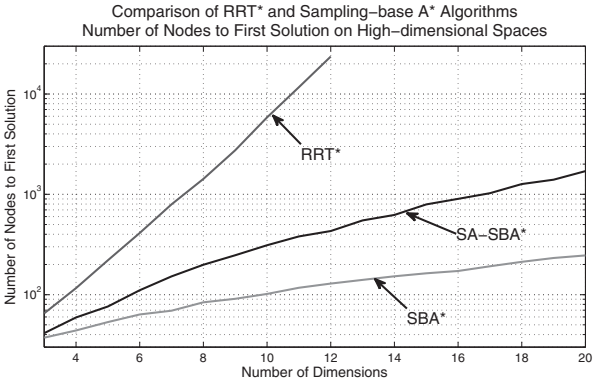


Figure 7. Monte Carlo results comparing RRT*, SBA*, and SA-SBA* on high-dimensional empty spaces, showing the mean number of vertices required to find the first solution versus the number of dimensions of the space.

4.2 High-dimensional empty spaces

Another interesting aspect to investigate is the behavior of the algorithm as the dimensionality of the problem is increased. In this section, we want to shed some light on the dimensionality alone and, thus, we use "empty" spaces as the configuration space, i.e. a high-dimensional Euclidean space (from 3 to 20 dimensions) with no obstacles, bounded with a unit hyper-cube and with a start and goal point on opposite corners of the hyper-cube. The heuristic function is, again, the Euclidean norm between the sample and the goal position, as in Equation (50). We limit the size of any segment (edge) connecting two configuration points to a length of $0.2\sqrt{N}$ where N is the number of dimensions in the space, i.e. there is always a minimum of 5 segments required to travel from the starting point to the goal point in a straight line.

Figure 7 shows the mean number of vertices required to reach an initial solution by all three algorithms for 100 runs. As expected, the RRT* algorithm shows an exponential relationship to the number of dimensions, as per the well-known *curse of dimensionality*. The RRT* algorithm seems to require approximately twice the amount of vertices for every additional dimension, i.e. the mean node count is $O(\gamma^N)$ with $\gamma = 1.92$. By contrast, if we direct our attention to the SBA* algorithms, we see sub-exponential relationships. Indeed, the SBA* results show a linear relationship between the mean node count and the dimensionality of the space, with the best regression giving $O(N^\rho)$ with $\rho = 1.02$. The SA-SBA* results are more difficult to characterize or find a good regression for, but it appears the relationship is quadratic, with a power-function regression giving $O(N^\rho)$ with $\rho = 2.01$, and the quadratic curve fit having the least residual error of all regressions tried. It is reasonable to conclude that the SA-SBA* algorithm achieves a mean node count that is bounded-above by the RRT* performance and bounded below by the SBA* performance in the absence of obstacles. Where exactly the SA-SBA* performance lies will naturally depend on the tuning of the simulated annealing schedule (e.g. initial temperature or cooling formula).

The linear relationship between the required node count and the dimensionality of the space when running the SBA* algorithm is interesting and deserves additional clarifications. For an unbiased, exploratory algorithm such as RRT*, the volume of space to be explored is an exponential function of the dimensionality, and so is the number of nodes needed to explore it. In a heuristically driven algorithm such as SBA*, the volume of "useful" space to be explored is limited to a corridor between the start and the goal points, in the absence of obstacles, i.e. when the space is η-entangled with a low value of η. The volume of this corridor is proportional to the length of the central path. The same phenomenon is true of A*, and does not depend on the relaxation of the heuristic. This phenomenon was also reported for the Guided-EST (Phillips et al., 2004). It is important to note, however, that the introduction of obstacles and, thus, raising the η value that characterizes the entanglement of the problem will expand the volume to be explored to find a solution, ultimately requiring a full exploration, as the RRT* does.

4.3 7-dof Manipulator

The target application of the proposed algorithm is an uncooperative satellite capture scenario involving a robotic manipulator mounted on a servicing spacecraft. As an uncontrolled satellite is drifting in the vicinity of the servicing spacecraft after an orbital rendezvous, the robotic manipulator should be able to autonomously move to capture the satellite while avoiding a self-collision or a collision with the target satellite. In this section, we show that the SBA* algorithms can be successfully applied to solve problems in this real scenario for a static target. What makes the SBA* algorithm a suitable candidate in this scenario is that the navigation problem is characterized by a rather straight-forward path to the target location while side-stepping a danger zone around the target's collision geometry, with possibly narrow corridors to traverse.

Our simulations of the satellite capture task are based on the experimental facility available at the Aerospace Mechatronics Laboratory at McGill University which uses a neutrally buoyant airship to emulate the target satellite and a robotic manipulator on a 3 m linear track to represent the space manipulator (see Figure 8(a)). The capture target for the end-effector of the manipulator is a single grapple fixture mounted on the airship which itself is a spherical blimp of 6 foot in diameter. The mobile manipulator system has a total of 7 dofs (6-dof manipulator and the linear track), and planning is performed in the 14-dimensional joint state-space, i.e. using the 7 joint positions and 7 joint velocities as the state representation stored in the motion graph and using per-joint cubic-spline interpolations between states. Collision geometries are represented by simple 3-dimensional primitives, and proximity queries (collision detection) are performed with simple closed-form expressions for each pair of primitives. Figure 8(b) shows a three-dimensional rendering of the planning environment.

Downloaded from ijr.sagepub.com by guest on January 20, 2015

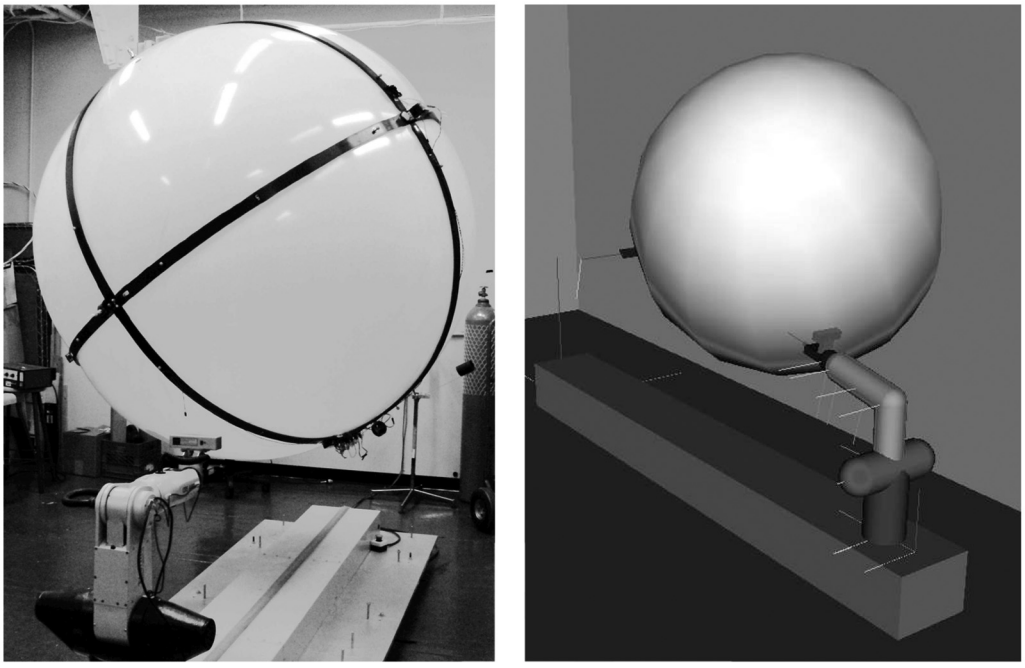


Figure 8. Picture of the robot-airship facility at the Aerospace Mechatronics Laboratory, McGill University: (a) actual picture; (b) virtual environment.

To normalize the dimensions of the state space, each component is divided by its maximum allowable rate of change, that is, joint positions are divided by the maximum joint velocities and the joint velocities are divided by the maximum joint accelerations, which results in each state component being represented as the time required to reach that position or velocity from the origin or from rest, respectively. We refer to this representation as the *reach-time representation*, expressed in seconds. The distance metric used is the $\mathcal{L}_\infty$-norm over the Euclidean norm of joint-wise position–velocity pairs, which approximates the time required for the most distant joint to be brought to the destination position and velocity. According to the distance metric, which is, again, used also as the travel cost, the heuristic function used for any sample u with associated point $p(u)$ is as follows:

$$h_{cs}(u) \equiv \max_{i=1\ldots7} |p_{goal,i} - p_i(u)| \tag{51}$$

$$h_{ss}(u) \equiv \max_{i=1\ldots7} \sqrt{(p_{goal,i} - p_i(u))^2 + (p_{goal,i+7} - p_{i+7}(u))^2} \tag{52}$$

where the subscripts cs and ss denote the configuration-space and state-space planners, respectively. The reach-time representations of the joint positions $p_{1\ldots7}$ and the joint velocities $p_{8\ldots14}$ are defined as

$$p_i \equiv \frac{q_i}{v_{max,i}}, \quad p_{i+7} \equiv \frac{v_i}{a_{max,i}}, \quad \text{for } i \in [1,7] \tag{53}$$

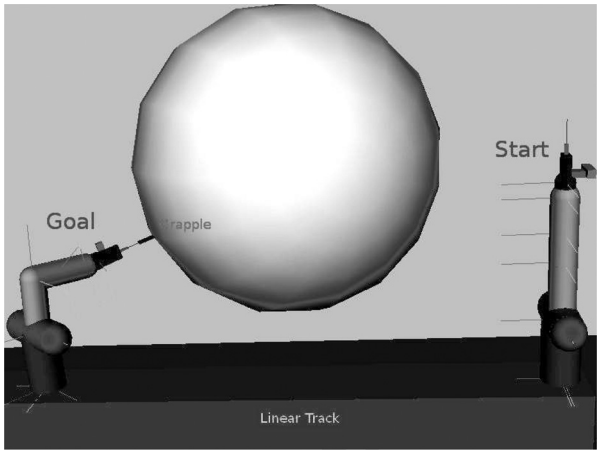


Figure 9. Static planning scenario on the virtual environment, showing the start and goal configurations.

where q_i is the joint coordinate i (angular or linear), v_i is the joint velocity i, and v_{max} and a_{max} are the maximum joint velocities and accelerations, respectively. The resulting unit of all values in $p(u)$ are seconds. In the planning scenarios, the maximum travel distance for any edge of the graph was set to 1 second (i.e. the radius of local reachable regions).

Figure 9 shows the start and goal configuration of the scenario that will be used to test the proposed algorithms. The scenario places the airship at an offset from the center of the track, leaving just enough room for the manipulator to move over to the opposing side of the track, which it is

Downloaded from ijr.sagepub.com by guest on January 20, 2015

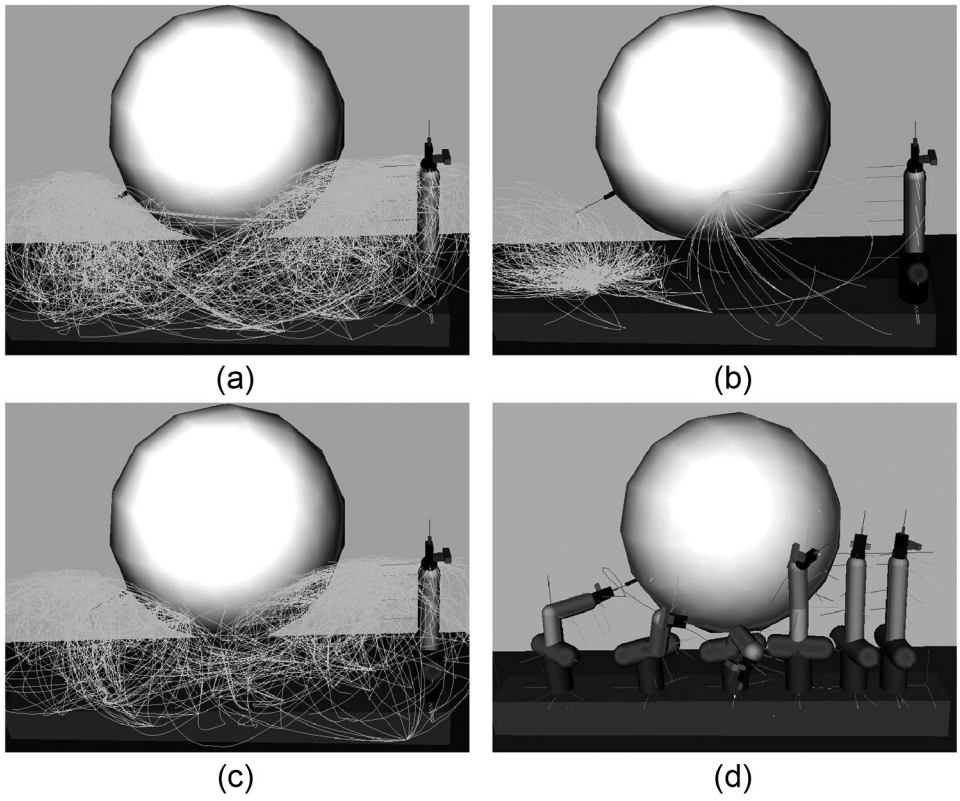


Figure 10. Resulting motion graphs after 1000 iterations on the robot-airship facility, shown as traces of the end-effector positions for (a) RRT*, (b) SBA*, and (c) SA-SBA*, and a still animation of the SA-SBA* solution in (d).

required to do because the grapple fixture, and thus the target configuration, is on that side. The main features of this scenario is that the corridor along which to travel to move from one side of the track to the other is quite narrow and requires the manipulator to be turned to the side (profile) and tilted away from target to allow a safe (collision-free) passage across.

As an example, Figure 10 shows the resulting motion-graphs from running the RRT*, SBA*, and SA-SBA* algorithms for 1000 iterations (1000 nodes). One such solution path is illustrated as a still animation in Figure 10(d), which shows clearly that the manipulator does indeed make considerable motion to avoid colliding with the airship on its way to the capture configuration.

4.3.1 Monte Carlo runs. To further validate the proposed methods in this real scenario, we present the results of Monte Carlo runs of the different algorithms for planning in the configuration space (zeroth order) and in the state space (first order) of this 7 degrees-of-freedom manipulator, amounting to a 7-dimensional space and a 14-dimensional space, respectively. The results presented in this section are accumulated over 1000 runs of the different algorithms. For configuration-space planning, the interpolation (or local planner) is linear and the distance metric used is the $\mathcal{L}_\infty$-norm of the reach-time representation of the joint positions or angles. For state-space planning, the interpolation is cubic and the distance is taken as the $\mathcal{L}_\infty$-norm of the joint-wise Euclidean norm of position and velocity, in reach-time representation. In these experiments, the initial relaxation factor for the anytime heuristic was 5.0, and the initial temperature for the simulated annealing schedule was 5.0.

Figures 11(a) and 11(b) show the average solutions found after a given number of iterations of the different algorithms. The results corroborate those obtained in the two-dimensional cases, as do the success rates seen in Figures 11(c) and 11(d). We can see that the SA-SBA* algorithm out-performs the RRT* algorithm on early solutions and then achieves a comparable asymptotic behavior. On the state-space planning (14 dimensions), the difference is the most significant, in favor of the SA-SBA* algorithm. Moreover, the reliability, in terms of success rates, of the SA-SBA* is also comparable to that of the RRT* algorithm in these scenarios. However, the SBA* algorithm does suffer from a significant reliability problem, especially when the dimensionality of the problem is increased to 14 in the state-space planning. Clearly, in very high-dimensional problems, the SBA* algorithm has more difficulty circumventing the obstacles and dead-ends due to the higher number of samples required to sufficiently increase the density values. Nevertheless, it is clearly beneficial to mix SBA* iterations into RRT* iterations, as demonstrated by the SA-SBA* algorithm's performance.

Table 1 presents an additional view of the results of the Monte Carlo runs. First, we have the average first solutions

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Table 1. First solution statistics for the robot-airship capture scenario, comparing RRT*, SBA* and SA-SBA*, in a zeroth-order space (configuration space, 7 dimensions) and a first-order space (state space, 14 dimensions).

Algorithm	Solution cost		Number of nodes		Execution time (s)
Zeroth-order (7D)	Mean	SD	Mean	SD	for 3000 nodes
RRT*	3.68	0.36	150	408	4.58
SBA*	3.66	0.31	289	485	3.17
SA-SBA*	3.67	0.34	152	333	3.99
First-order (14D)	Mean	SD	Mean	SD	for 6000 nodes
RRT*	5.18	0.46	1685	1385	13.83
SBA*	5.21	0.61	2750	1710	4.91
SA-SBA*	5.09	0.44	1810	1370	10.30

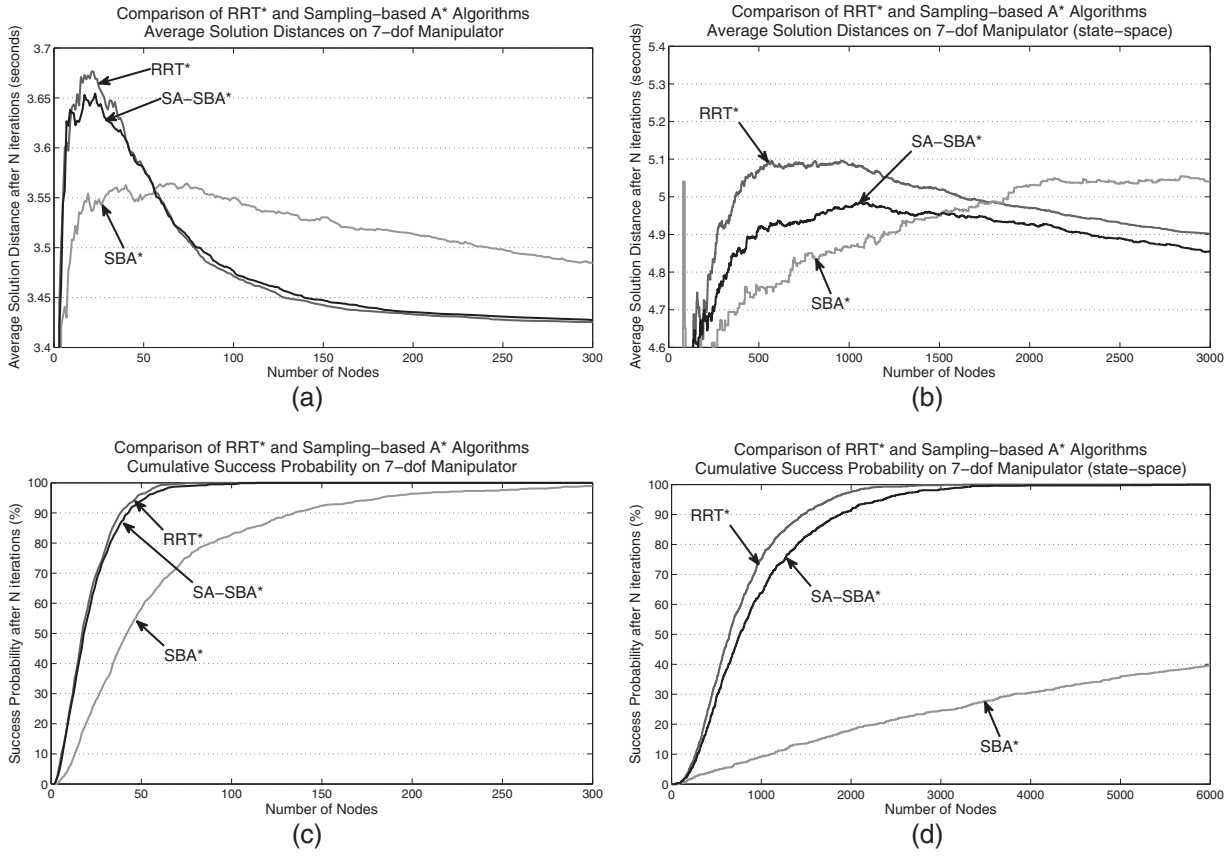


Figure 11. Monte Carlo results comparing RRT*, SBA*, and SA-SBA* for the robot-airship capture scenario in a zeroth-order space (position only, 7 dimensions) and a first-order space (state-space, 14 dimensions), showing the average cost of solutions (a), (b) and the cumulative success rates (c), (d).

found by the planners on the 1000 runs. We can see that all algorithms produce comparable results, with only marginal improvements on the mean value and the standard deviation for the SA-SBA* algorithm. Then, we reported the average number of nodes required to find the first solution for each planner. We can observe that SBA* algorithm is significantly less reliable, but the SA-SBA* recoups that reliability and even achieves lower standard deviation than the RRT* algorithm. Finally, the table shows the running time, in seconds, required to run 3000 and 6000 iterations for the zeroth-order and first-order planning problem, respectively. By comparing the results, we can see that the SBA* is significantly faster than RRT*, and that the SA-SBA* falls in between, as expected. The significant runtime reductions achieved by SBA* iterations must be due to the fact that it requires only a single nearest-neighbor query as opposed to two for RRT* iterations, since the only other significant computational cost is the collision-checking, which we would expect to incur the same cost in all algorithms tested.

Downloaded from ijr.sagepub.com by guest on January 20, 2015

5. Conclusion

In this paper, we have proposed a new class of sampling-based path-planning algorithms derived from a formal generalization of the classic A* algorithm. The key generalization steps involve casting the value of the nodes of the motion graph in terms of a probabilistic expected value, and then replacing the binomial concept of explored versus unexplored nodes with a metric for the density of the region to which the node belongs. Furthermore, salient formal analysis was brought forth that demonstrates the probabilistic completeness and convergence of an idealized version of the proposed algorithm. A convenient, recursive formulation for the density metric was also presented in the form of an approximation of the KL divergence between a Gaussian distribution and mixture of the Gaussian distributions over neighboring nodes.

Then, a number of practical considerations and improvements were presented that fall in line with existing literature in sampling-based path-planning. A final novelty proposed here was the introduction of a simulated annealing schedule, a technique borrowed from nonlinear optimization and machine learning literature, as a means to balance between exploratory and exploitative steps, which was then applied to combine the RRT* and SBA* algorithms, respectively.

The results presented in this paper support the claims made throughout the development of the proposed methods. First, it was shown that the SBA* algorithms can more effectively explore the region around the optimal path, improving the success rates, the quality of the solutions found and the rate at which they can be improved during the later phase of the algorithms. These are aspects not only desirable from a quality stand-point but also leading to measurable benefits in time and computational capital spent towards finding a solution to a path-planning problem.

Second, it was observed that the interleaving of exploratory iterations (RRT*) with exploitative iterations (SBA*) via a simulated annealing schedule improves the performance, in most aspects, compared with either algorithms in isolation. This suggests, or confirms, that striking a correct balance *between* exploration and exploitation is the key to a path-planner that performs well in the type of scenarios considered in this paper.

Third, the behavior of the SBA* algorithms in high-dimensional spaces shows not only a graceful degeneration of the algorithms in scenarios in which a simple straight-line interpolation would suffice, but that the average number of nodes required to solve a problem is only linearly dependent on the dimensionality of the space under obstacle-free scenarios. However, the presence of obstacles in the configuration space has a definite adverse effect on the performance of the SBA* algorithm: a problem that seems to vanish when RRT* iterations are scheduled into the SBA* algorithm with, as proposed in this paper, a simulated annealing schedule. This also suggests a possible future line of investigation on the effect of the guiding heuristic function and the depth of the local minimas it generates on the performance of the nominal SBA* algorithm, since these factors are known to significantly degrade the performance of the A* algorithm.

Finally, the proposed class of algorithms has demonstrated satisfactory performance on a real scenario of autonomous capture of a large free-floating object with a robotic manipulator, i.e. a high-dimensional space (14 dimensions) with complex collision geometries. The statistical analysis of those real high-dimensional scenarios demonstrates that the qualities previously observed of the proposed methods do, indeed, carry over to those difficult problems.

Acknowledgments

This work was made possible with the support of the Vanier Canada Graduate Scholarship from the National Science and Engineering Research Council of Canada (NSERC), and the support of the Hydro Quebec Doctoral Award as part of the McGill Engineering Doctoral Awards (MEDA). The authors would like to thank the reviewers of this article for providing very constructive feedback and for upholding the high standards of rigor required for productive scientific discourse.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Notes

1. Square brackets denote an association of precomputed values as opposed to a computation of those values.
2. See https://github.com/mikael-s-persson/ReaK, last accessed on 19 February 2014.

References

Belghith K, Kabanza F, Hartman L and Nkambou R (2006) Anytime dynamic path-planning with flexible probabilistic roadmaps. In: *Proceedings of the 2006 IEEE international conference on robotics and automation (ICRA'06)*, Orlando, FL, pp. 2372–2377.

Berg JVD, Ferguson D and Kuffner JJ (2006) Anytime path planning and replanning in dynamic environments. In: *Proceedings of the 2006 IEEE international conference on robotics and automation (ICRA'06)*, Orlando, FL, pp. 2366–2371.

Branicky MS, LaValle SM, Olson K and Yang L (2001) Quasi-randomized path planning. In: *Proceedings of the 2001 IEEE international conference on robotics and automation (ICRA'01)*, vol. 2, Seoul, Korea, pp. 1481–1487.

Burns B and Brock O (2005) Sampling-based motion planning using predictive models. In: *Proceedings of the 2005 IEEE international conference on robotics and automation (ICRA'05)*, Barcelona, Spain, pp. 3120–3125.

Burns B and Brock O (2007) Single-query motion planning with utility-guided random trees. In: *Proceedings of the 2007 IEEE international conference on robotics and automation (ICRA'07)*, Roma, Italy, pp. 3307–3312.

Chowdhury RA (2007) *Algorithms and Data Structures for Cache-efficient Computation: Theory and Experimental Evaluation*. PhD Thesis, University of Texas at Austin.

Downloaded from ijr.sagepub.com by guest on January 20, 2015

Ferguson D, Kalra N and Stentz A (2006) Replanning with RRTs. In: *Proceedings of the 2006 IEEE international conference on robotics and automation (ICRA'06)*, Orlando, FL, pp. 1243–1248.

Fu AWC, Chan PMS, Cheung YL and Moon YS (2000) Dynamic VP-tree indexing for *N*-nearest neighbor search given pairwise distances. *The Very Large Data Bases Journal (VLDB)* 9: 154–173.

Geraerts RJ and Overmars MH (2002) *A Comparative Study of Probabilistic Roadmap Planners*. Technical Report, Utrecht University: Information and Computer Science, Utrecht, The Netherlands.

Gonzalez JP and Likhachev M (2011) Search-based planning with provable suboptimality bounds for continuous state spaces. In: *Proceedings of the fourth international symposium on combinatorial search (SoCS'11)*, Castell de Cardona, Barcelona, Spain. AAAI Press, pp. 60–67.

Hart PE, Nilsson NJ and Raphael B (1968) A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* 4(2): 100–107.

Hershey JR and Olsen PA (2007) Approximating the Kullback–Leibler divergence between Gaussian mixture models. In: *IEEE international conference on acoustics, speech and signal processing*, volume 4, Honolulu, HI, pp. 317–320.

Hsu D (2000) *Randomized Single-Query Motion Planning in Expansive Spaces*. PhD Thesis. Department of Computer Science, Stanford University, Stanford, CA, pp. 71–90.

Hsu D, Latombe JC and Motwani R (1999) Path planning in expansive configuration spaces. *International Journal of Computational Geometry & Applications* 09(4/5): 495–512.

Ingber L (1996) Adaptive simulated annealing (ASA): lessons learned. *Journal of Control and Cybernetics* 25: 33–54.

Jamriska O, Sykora D and Hornung A (2012) Cache-efficient graph cuts on structured grids. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR'12)*. IEEE Computer Society Press, pp. 3673–3680.

Karaman S and Frazzoli E (2011) Sampling-based algorithms for optimal motion planning. *The international journal of robotics research* 30(7): 846–894.

Karaman S, Walter MR, Perez A, Frazzoli E and Teller S (2011) Anytime motion planning using the RRT*. In: *IEEE international conference on robotics and automation (ICRA)*, Shanghai, China, pp. 1478–1483.

Kasheff Z (2004) *Cache-Oblivious Dynamic Search Trees*. M.Eng. Thesis, Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology.

Kavraki LE and Latombe JC (1998) Probabilistic roadmaps for robot path plannings. In: *Motion Planning in Robotics*. New York: John Wiley & Sons.

Kavraki LE, Svestka P, Latombe JC and Overmars MH (1996) Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation* 12(4): 566–580.

Kuffner JJ and LaValle SM (2000) RRT-Connect: an efficient approach to single-query path planning. In: *Proceedings of the 2000 IEEE international conference on robotics and automation (ICRA'00)*, vol. 2, San Francisco, CA, pp. 995–1001.

Kullback S and Leibler RA (1951) On information and sufficiency. *Annals of Mathematical Statistics* 22(1): 79–86.

LaValle SM (1998) *Rapidly-exploring Random Tree: A New Tool for Path Planning*. Technical Report 98-11, Department of Computer Science. Iowa State University.

LaValle SM and Kuffner JJ (2001) Randomized kinodynamic planning. *The International Journal of Robotics Research* 20: 378–400.

Likhachev M, Ferguson D, Gordon G, Stentz A and Thrun S (2005) Anytime dynamic A*: An anytime, replanning algorithm. In: *Proceedings of the international conference on automated planning and scheduling (ICAPS)*, Monterey, CA. AAAI Press, pp. 262–271.

Likhachev M, Gordon G and Thrun S (2003) ARA*: Anytime A* with provable bounds on sub-optimality. In: *Proceedings of the 2003 conference in advances in neural information processing systems (NIPS)*, Whistler, BC, Canada. Cambridge, MA: MIT Press, pp. 767–774.

Phillips JM, Bedrossian N and Kavraki LE (2004) Guided expansive spaces trees: a search strategy for motion- and cost-constrained state spaces. In: *Proceedings of the 2004 IEEE international conference on robotics and automation (ICRA'04)*, vol. 4, New Orleans, LA, pp. 3968–3973.

Plaku E (2012) Guiding sampling-based motion planning by forward and backward discrete search. In: Su CY, Rakheja S and Liu H (eds.), *Intelligent Robotics and Applications* (Lecture Notes in Computer Science, vol. 7508). Berlin: Springer, pp. 289–298.

Rickert M, Brock O and Knoll A (2008) Balancing Exploration and Exploitation in Motion Planning. In: *IEEE international conference on robotics and automation (ICRA)*, Pasadena, CA, pp. 2812–2817.

Sanchez G and Latombe JC (2001) A single-query bi-directional probabilistic roadmap planner with lazy collision checking. In: Jarvis R and Zelinsky A (eds.), *International Symposium on Robotics Research* (Springer Tracts in Advanced Robotics, vol. 6). Berlin: Springer, pp. 403–417.

Sucan I and Kavraki L (2012) A sampling-based tree planner for systems with complex dynamics. *IEEE Transactions on Robotics* 28(1): 116–131.

Sucan IA and Kavraki LE (2010) On the implementation of single-query sampling-based motion planners. In: *Proceedings of the 2010 IEEE international conference on robotics and automation (ICRA'10)*, Anchorage, AK, pp. 2005–2011.

Vazquez-Otero A, Faigl J and Munuzuri AP (2012) Path planning based on reaction–diffusion process. In: *IEEE/RSJ international conference on intelligent robots and systems*, Vilamoura, Algarve, Portugal, pp. 896–901.

Zucker M, Ratliff N, Dragan A, et al. (2013) CHOMP: Covariant Hamiltonian Optimization for Motion Planning. *The International Journal of Robotics Research* 32(11): 1164–1193.

Downloaded from ijr.sagepub.com by guest on January 20, 2015